Adversarial Prototype Learning for Hyperspectral Image Classification

Shuai Wang, *Graduate Student Member, IEEE*, Bo Du, *Senior Member, IEEE*, Dingwen Zhang, *Member, IEEE*, and Fang Wan, *Member, IEEE*

***Abstract*— In hyperspectral image (HSI) classification, the training set often contains a very limited number of high-dimensional samples, which can cause overfitting problems, especially in deep learning (DL) frameworks. This situation worsens when a bias exists between the feature distributions of the training and testing sets. In this article, we propose a novel method, referred to as adversarial prototype learning (APL), for learning an accurate HSI classification model in a uniform manner when the training set contains few, high-dimensional, and biased samples. APL consists of a prototype learning module (PLM) and an adversarial alignment module (AAM). The PLM aims to alleviate overfitting by training prototypical classifiers with a simple inductive bias in the initial feature space. The AAM aims to reduce the bias between the feature distributions of the training and testing sets using two adversarial prototypical classifiers learned by the PLM. Iteratively training the PLM and AAM results in alignment of the feature distributions between the training and testing sets while improving the generalization ability of the prototypical classifiers. The theoretical analysis indicates that APL is able to lower the upper error bound when classifying testing samples. We further apply APL in a DL framework to establish the adversarial prototypical network (APNet) architecture. Experimental results on four publicly available HSI datasets demonstrate that the proposed APNet alleviates overfitting, aligns the feature distributions between the training and testing sets, and achieves state-of-the-art performance.**

***Index Terms*— Adversarial learning, feature distribution alignment, hyperspectral image (HSI) classification, prototype learning.**

I. Introduction

HYPERSPECTRAL images (HSIs) contain hundreds of electromagnetic spectra with wavelengths ranging from the visible light region to the near-infrared. Due to the abundant and detailed information contained in HSIs, they have been widely used in geoscience and remote sensing studies [1]–[4], in which HSI classification is one of the most fundamental and commonly addressed tasks. HSI classification methods usually treat each pixel in an HSI as a sample and classify each sample into a predefined category based on the image content.

Many efforts related to HSI classification rely on extracting representative features for each sample and then correctly classifying them. Such methods can be divided into conventional machine-learning-based and deep-learning-based methods. Conventional machine-learning-based methods use raw pixels or local patches to extract handcrafted features [5]–[11] and classify them using classifiers, such as support vector machines (SVMs) [12] or logistic regression [13]. Recently, in the fields of computer vision and natural language processing [14]–[16], deep learning (DL) has shown enormous advantages over conventional machine learning. Inspired by this, an increasing number of HSI classification methods [17] have adopted autoencoders [18], recurrent neural networks (RNNs) [19] or convolutional neural networks (CNNs) [20]–[24] to extract DL features when classifying HSI samples.

Compared with autoencoder-based methods [18] and RNN-based methods [19], which extract only the spectral features of single pixels, CNN-based methods often convert the training samples from pixels into local HSI patches [20]–[23] or a global HSI patch (i.e., a patch-free form) [24]. These methods use convolutional layers to capture the spatial information of each training sample, improving the feature representations with the help of unlabeled/testing pixels in the training patches. In addition, generative adversarial networks (GANs) [25] can be used to learn the feature distributions of HSIs by utilizing the rich information available from unlabeled samples and generating virtual training samples. Nevertheless, these methods are not based on the assumption that the classifier should have a simple inductive bias and, therefore, fail to alleviate the overfitting problem that can arise when the training set is small [see Fig. 1(a)]. Furthermore, these methods ignore the bias between the training and testing sets, which hinders such a model from learning representative features for the testing samples. The problem of improving the generalization ability of an HSI classification model trained on a small training set remains to be explored. This problem is particularly serious in classification tasks with large feature dimensions and very few training samples, which are widely encountered in many computer vision tasks [26], [27].

Manuscript received April 14, 2021; revised July 8, 2021; accepted July 21, 2021. Date of publication August 9, 2021; date of current version January 17, 2022. This work was supported in part by the China National Postdoctoral Program for Innovative Talents under Grant BX20190323 and in part by the National Natural Science Foundation of China under Grant 62006216. *(Corresponding author: Fang Wan.)*

Shuai Wang is with the School of Earth Sciences and Resources, China University of Geosciences, Beijing 100083, China (e-mail: s.wang.738b5e05@gmail.com).

Bo Du is with the School of Computer Science, Wuhan University, Wuhan 430079, China (e-mail: gunspace@163.com).

Dingwen Zhang is with the Brain and Artificial Intelligence Laboratory, School of Automation, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: zhangdingwen2006yyy@gmail.com).

Fang Wan is with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: wanfang@ucas.ac.cn).

Digital Object Identifier 10.1109/TGRS.2021.3100496

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

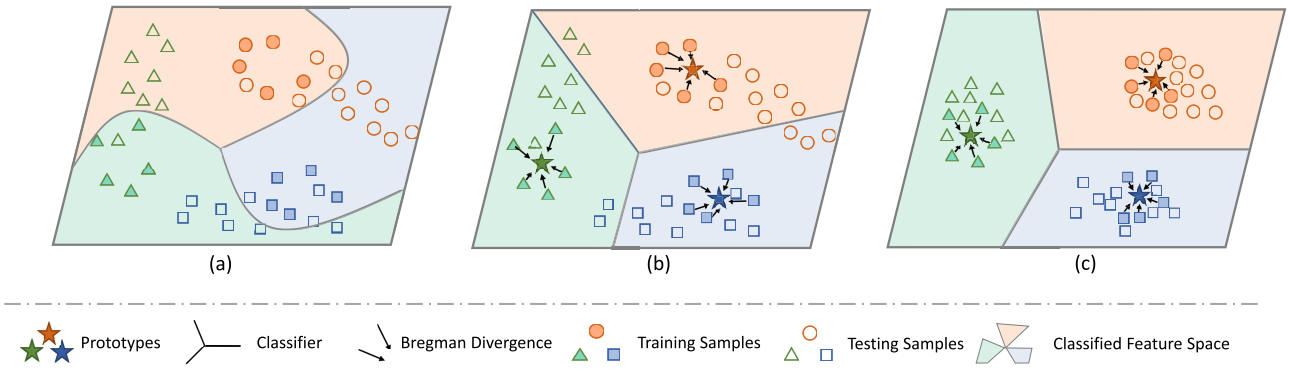


Fig. 1. Illustration of the HSI classification results of (a) conventional DL-based HSI classifier, (b) prototypical HSI classifier, and (c) APL (ours).

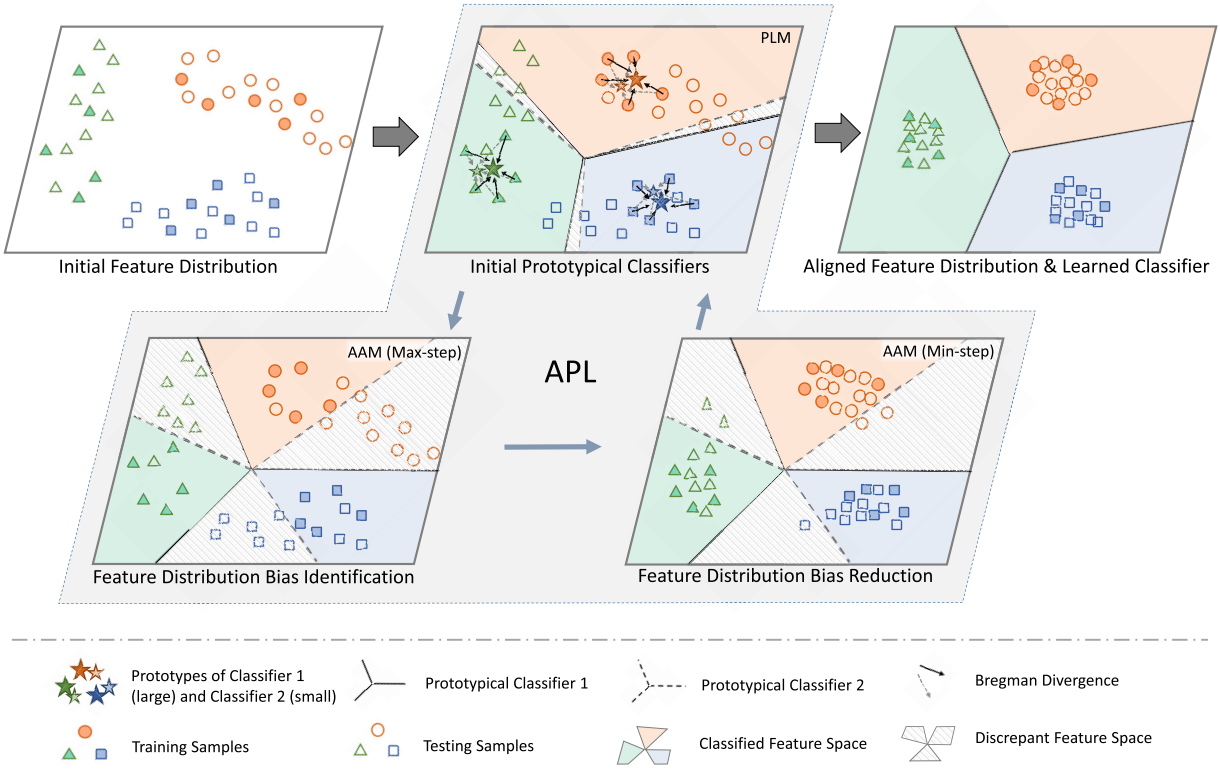


Fig. 2. Illustration of the proposed APL method. The "classified feature space" refers to the feature space in which the two classifiers agree with each other, while they disagree with each other in the "discrepant feature space."

In this article, we propose a novel method, referred to as adversarial prototype learning (APL), for learning an accurate HSI classification model in a uniform manner when the training set contains few, high-dimensional, and biased samples. APL consists of a prototype learning module (PLM) and an adversarial alignment module (AAM). The PLM is motivated by prototype learning [26] and aims to learn prototypical classifiers with a simple inductive bias in the feature space of the training set to alleviate overfitting. Nevertheless, the bias between the training and testing sets will hinder a classifier from generalizing well to the testing set [see Fig. 1(b)]. The AAM is, thus, proposed to align the feature distributions by utilizing two prototypical classifiers learned by the PLM in an adversarial manner. The AAM learns a feature space based on both the training and testing sets, in which the distribution bias between the training and testing sets is reduced. The PLM is then trained again in this feature space to improve the generalization ability of the classifiers. In this way, the PLM and AAM are iteratively trained several times until the feature distributions are aligned, while classifiers with a good generalization ability are learned [see Fig. 1(c)].

To train the PLM, we fine-tune the parameters of the model by minimizing the sample classification losses to learn a feature space and prototypes for each class based on the training set. A metric space is then established using a Bregman divergence [28], in which samples can be classified by calculating their feature distances (i.e., the Bregman divergences) to the prototypes (see Fig. 2). To train the AAM, we first fix the parameters in the feature extractor such that the feature space remains unchanged. The classifiers are fine-tuned to maximize

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

the discrepancy between their predictions on the testing set and minimize the sample classification loss on the training set. Consequently, the testing samples, which are not aligned with the training set, can be identified from their large prediction discrepancy (see Fig. 2, Max-step). Then, we fix the classifiers and fine-tune the feature extractor to minimize the prediction discrepancy of the classifiers. The bias between the identified testing samples and the training samples is, thus, reduced (see Fig. 2, Min-step). We further present a theoretical analysis of APL to show that, in this way, the upper error bound when classifying testing samples can be reduced.

To implement APL in a DL framework, we further propose the adversarial prototypical network (APNet) architecture. In APNet, the feature extractor includes four residual squeeze-and-excitation (ResSE) blocks, each of which consists of a group of convolutional layers with BN and rectified linear unit (ReLU) layers. The sample classifiers have the same architecture, but they own independent parameters and prototypes. The proposed APNet, which directly receives an HSI as input and outputs the corresponding class prediction map, has been efficiently trained in an end-to-end manner.

The main contributions of this article can be summarized as follows.

1) We propose a novel method, referred to as APL, in which a PLM and an AAM are iteratively trained to learn prototypical classifiers with a simple inductive bias and align the feature distribution of the training and testing sets, thus alleviating the overfitting problem.
2) We present a theoretical analysis of APL and show that APL can reduce the upper bound of the error when classifying testing samples.
3) We propose the APNet architecture to implement APL in a DL framework. The proposed APNet is able not only to extract spatial and spectral features by means of well-designed ResSE blocks but also to establish long-range spatial feature dependencies, and it can be efficiently trained in an end-to-end manner.
4) The proposed APNet significantly outperforms state-of-the-art methods on four publicly available HSI datasets.

In what follows, we first introduce related works in Section II. The proposed methodology of APL is presented in Section III. We then introduce the design of APNet for implementing APL in a DL framework in Section IV. Experimental results and analysis are given in Section V. We finally conclude this article in Section VI.

II. Related Works

The proposed method is related to HSI classification methods with limited training samples, unsupervised domain adaptation (UDA) methods, and semisupervised methods.

A. HSI Classification Methods

Conventional methods based on handcrafted features consider the spectral features of each individual pixel of an HSI and use these features to train SVM [12], multinomial logistic regression [13], Markov random field (MRF) [29], or artificial neural network (ANN) [30] classifiers. Because the dimensionality of the spectral features is usually large, while the number of training samples is small, these methods often suffer from overfitting on the training set. To solve this problem, the linear discriminant analysis (LDA) [5], the independent component analysis (ICA) [6], and the principal component analysis (PCA) [7] may be used to reduce the data dimensions and extract compact spectral features. However, these methods fail to utilize spatial information among samples. Researchers have, therefore, used morphological profiles [8], wavelets [9], local binary patterns (LBPs) [10], and Gabor filters [11] to extract spatially aware features based on the spatial context of each sample.

Inspired by the excellent performance of DL [14]–[16] in feature extraction, an increasing number of DL-feature-based HSI classification methods have been proposed. An RNN-based method [19] is used to extract sequential spectral features of HSI pixels. CNN-based methods [20], [21] learn the spatial features of local HSI patches. A 3-D-CNN-based method [22] has been proposed to make both spectral and spatial features explicitly learnable. The feature pyramid approach [23], [31], [32] has been adopted to extract multiscale features of HSI patches. A squeeze-and-excitation network (SENet) [24] has been adopted to capture the spectral attention information of HSI images. An attention multibranch CNN equipped with adaptive region search [33] has been proposed to selectively extract joint spatial–spectral features and exploit complementary information among different layers. GANs [34] have been used, on the basis of collaborative learning and an attention mechanism, to greatly improve the feature representations for both spatial and spectral information. A nonlocal graph convolutional network (GCN) [35] has been proposed to extract global spatial features from the whole image rather than from single pixels or patches of an HSI.

However, the questions of how to alleviate the overfitting problem and how to align the feature distributions of the training and testing sets in a DL framework remain challenging issues, and these questions are the subjects of this article.

B. Unsupervised Domain Adaptation Methods

UDA aims to minimize the classification risk when applying a model trained on a labeled source domain to an unlabeled target domain. UDA has been extensively explored in image classification tasks [36]–[38].

UDA methods mainly focus on aligning the data or feature distributions in the source domain and the target domain. A few methods [39]–[42] align the domains based on a subspace. A PCA-based method [39] has been proposed to align the data distributions in such a subspace using a matrix transformation. In the method of [40], CNN is first used to extract features that are treated as tensors, and the Tucker decomposition is further used to achieve an invariant subspace. An ensemble based on the supervised multiview canonical correlation analysis (CCA) [41] has been proposed with the aim of solving the heterogeneous domain adaptation issue. Cross-domain collaborative learning [42] has been proposed,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

with the help of cluster CCA and a random walker, for aligning source and target domains. Other methods focus on reducing the distance or domain divergence between the source and target domains. The maximum mean discrepancy (MMD) [43] has been proposed as a distance metric to minimize the domain divergence in a reproducing kernel Hilbert space (RKHS) [44], [45]. Progressive domain distance minimization [46] has been achieved by mining pseudolabels for model fine-tuning. Bousmalis *et al.* [47] and Hoffman *et al.* [48] attempted to reduce the domain divergence by taking advantage of adversarial and generative models to confuse the domains while aligning the feature distributions. The cycle-consistent adversarial domain adaptation (CyCADA) method [48] transfers samples across domains at both the pixel and feature levels.

Recent domain adaptation methods for HSI classification are equipped with dictionary learning [49], deep neural networks [50], [51], and tensor representations [52]. However, UDA requires extra data from the other domain and suffers from overfitting on a small training set. In contrast, our method allows classifiers to be learned in a single domain and is able to alleviate overfitting while reducing the feature distribution bias between the training and testing sets.

C. Semisupervised Methods

Semisupervised methods are trained on a set of samples of which only a subset are labeled. The transductive SVM (TSVM) [53] method was first applied for HSI classification, with the aim of maximizing the classification margin on a sample set containing both labeled and unlabeled data. A self-learning technique with an embedding regularizer [54] has been adopted for low-density separation. Pseudolabels [55] of unlabeled data can be used for feature learning. GANs [56] and 3-D GANs [57], which are able to learn data distributions from unlabeled data, can be adopted for sample generation. A semisupervised stacked autoencoder (Semi-SAE) with a cotraining framework [58] and a graph-based method [59] have been proposed for HSI classification in recent years. Moreover, spatial–spectral methods [20]–[24], [31], [32] that use HSI patches can also be regarded as semisupervised methods, as they utilize the unlabeled data in the HSI patches.

Although these semisupervised methods use unlabeled samples for model training, the bias between the feature distributions of labeled and unlabeled samples is still ignored.

D. Prototype Learning Methods

Prototype learning is a conventional pattern recognition method. The earliest prototype learning method was K-nearest neighbors (K-NNs). Learning vector quantization (LVQ) [60] was later proposed to reduce the large storage space and computational requirements of K-NN and has been widely used in the recognition of handwritten numeral strings [61]–[63]. With the rise of DL, prototype learning has come to play an important role in few-shot visual recognition tasks [64], [65].

In recent years, prototype learning has been applied in remote sensing image recognition tasks. In [66], prototype learning was used to record spectral features using synergetics theory. In [67], unsupervised feature selection methods for features representing bands in the prototype space were proposed for band selection in HSIs. In [68], simultaneous sparse multimanifold learning (SSMML) was used as a prototype to model multimodal data structures. In [69], prototypical networks were first trained on base classes to learn representative features and then fine-tuned on novel classes with few-shot annotations. Although spatial–spectral prototypical networks (SSPNs) [70] and deep prototypical networks (DPNs) [71] have been introduced as prototypical networks for application to remote sensing images, they unfortunately ignore the bias between labeled and unlabeled data when the training set is very small.

III. Adversarial Prototype Learning

In this section, the proposed APL methodology is introduced. Preliminaries are first presented in Section III-A. The PLM is then introduced to alleviate the overfitting problem in Section III-B. The AAM is further proposed to align the feature distributions between the training and testing sets in Section III-C. Finally, we conduct analyses of APL in Section III-D.

A. Preliminaries

For the HSI classification task, an HSI with dimensions of $W \times H \times S$ is given, where W, H, and S denote the width, height, and the number of spectra, respectively. A small training set of $N = C \times K$ pixels in the HSI is labeled in accordance with their land cover classes, where C is the number of classes in the HSI and K is the number of labeled pixels for each class. Let $\mathcal{X}_L = \{x_1, x_2, \ldots, x_N\}$ denote the set of training samples, and let $\mathcal{Y}_L = \{y_1, y_2, \ldots, y_N\}$ denote the label set of $\mathcal{X}_L$. y_i is a $1 \times C$ vector that is one-hot encoded, where $y_{i,c} = 1$ indicates that sample x_i belongs to class c, $c \in \{1, 2, \ldots, C\}$, and $y_{i,c} = 0$ indicates that it does not.

To train an HSI classification model on the training set $\{(\mathcal{X}_L, \mathcal{Y}_L)\} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, a feature extractor g parameterized by θ_g is designed to convert the high-dimensional data point x_i into the compact feature $g(x_i, \theta_g)$. The extracted features are then fed into a classifier f, which predicts the classification probability of sample x_i. By minimizing the HSI classification loss, the optimal parameters θ_g and θ_f are learned, which can be used to further infer the labels of the unlabeled HSI samples $\mathcal{X}_U = \{x_1, x_2, \ldots, x_M\}$, where M is the number of samples in the unlabeled HSI sample set and $M \gg N$ is often established.

Conventional HSI classification models are simply trained on the small training set $\{(\mathcal{X}_L, \mathcal{Y}_L)\}$. Consequently, they can easily suffer from overfitting and ignore the bias between the feature distributions of the training and testing sets. To solve this problem, two components are desired to improve the generalization ability of the trained model: a classifier that is less likely to overfit on a small training set and a feature extractor that is able to extract representative features of the testing samples. To achieve this purpose, we introduce the APL method that consists of a PLM and an AAM.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

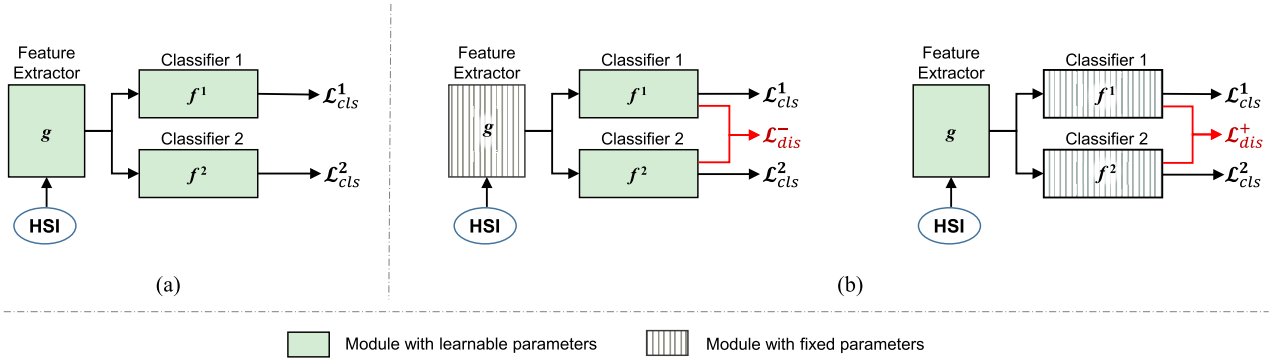


Fig. 3. Flowchart of the proposed APL method during model training. (a) PLM. (b) AAM.

The PLM is first applied to learn two prototypical classifiers, f^1 and f^2, with a simple inductive bias to alleviate the overfitting problem. The AAM utilizes the discrepancy between these prototypical classifiers to reduce the feature distribution bias between the training and testing sets in an adversarial manner. Through iterative training of the PLM and AAM, the proposed APL method is able to alleviate the overfitting problem while aligning the feature distributions of the training and testing sets.

B. Prototype Learning Module (PLM): Prototypical Classifier Acquisition

To alleviate the overfitting problem in HSI classification, we first train classifiers with a simple inductive bias on the training set $\mathcal{X}_L$ [see Fig. 3(a)]. As described in the previous subsection, the features are first extracted by g and then classified by f^1 and f^2 separately. Let $\mathbf{w}_c^1$ and $\mathbf{w}_c^2$ denote the prototypes of the cth class for the two prototypical classifiers. $\mathbf{w}_c$ is a vector that has the same dimensions as the feature vector of a sample. The class probability predicted by a prototypical classifier is defined as

$$p_c(x_i) = \exp\left(\frac{-||g(x_i, \theta_g) - \mathbf{w}_c||^2}{2R_c^2}\right) \tag{1}$$

where we set $\mathbf{w}$ to be learnable to find the most representative prototypes and R_c is also a learnable parameter indicating the rejection radius for class c. Accordingly, the parameters of the proposed classifiers are set to be $\theta_{f^1} = \{\mathbf{w}^1, R^1\}$ and $\theta_{f^2} = \{\mathbf{w}^2, R^2\}$. The HSI classification loss on the training set $\{(\mathcal{X}_L, \mathcal{Y}_L)\}$ is defined as

$$\mathcal{L}_{\text{cls}}(\mathcal{X}_L) = \frac{-1}{N} \sum_{x_i \in \mathcal{X}_L} \sum_c y_{i,c} \log p_c(x_i) + (1 - y_{i,c}) \log(1 - p_c(x_i)). \tag{2}$$

It can be seen in (1) that the feature of x_i is close to the prototype $\mathbf{w}_c$ only if x_i belongs to class c. In contrast, the features of those samples that do not belong to class c will be far away from the prototype $\mathbf{w}_c$. By combining (1) with (2) and minimizing (2), a metric space is learned for classification, as follows:

$$c^* = \arg\min_c p_c(x_i) = \arg\min_c \frac{||g(x_i, \theta_g) - \mathbf{w}_c||^2}{R_c^2} \tag{3}$$

where $c^* \in \{1, 2, \ldots, C\}$ is the predicted class of sample x_i. Equation (3) implies that the metric space is established on the basis of the scaled Euclidean distance, which is a regular Bregman divergence [28]. According to the theory in [26], prototype learning with a Bregman divergence can be interpreted as mixture density estimation, while a prototypical classifier learned with the Euclidean distance can be regarded as a linear model, with a very simple inductive bias. The prototypical classifiers learned by the PLM are, therefore, able to alleviate the overfitting problem.

The loss function of the PLM is then defined as

$$\mathcal{L}_{\text{PLM}}(\mathcal{X}_L) = \mathcal{L}_{\text{cls}}^1(\mathcal{X}_L) + \mathcal{L}_{\text{cls}}^2(\mathcal{X}_L) \tag{4}$$

by combining (2) with the two adversarial classifiers. When (4) is minimized, the discriminative features and the initial classifiers f^1 and f^2 are learned.

C. Adversarial Alignment Module (AAM): Feature Distribution Alignment

Although the PLM alleviates the overfitting problem by learning prototypical classifiers, it is trained based on the assumption that the feature distribution of the training set is representative of that of the testing set. Unfortunately, there often exists a bias between these feature distributions, which hinders the generalization to the testing set of classifiers learned on the training set. To solve this problem, we further propose the AAM to reduce the bias between the feature distributions in an adversarial manner.

1) Max-Step: Feature Distribution Bias Identification: To reduce the bias between the feature distributions of the training and testing sets, the biased features in the testing set should first be identified. The parameters θ_g of the feature extractor g should remain unchanged so that the feature space is fixed during the max-step. We then fine-tune the parameters θ_{f^1} and θ_{f^2} [see Fig. 3(b) (left)] to minimize the following loss function:

$$\mathcal{L}_{\text{Max}}(\mathcal{X}) = \mathcal{L}_{\text{cls}}^1(\mathcal{X}_L) + \mathcal{L}_{\text{cls}}^2(\mathcal{X}_L) + \lambda \mathcal{L}_{\text{dis}}^-(\mathcal{X}_U) \tag{5}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

where λ is an experimentally determined regularization parameter and $\mathcal{L}_{\text{dis}}^{-}$ is the discrepancy between the two adversarial classifiers' predictions on the testing set, expressed as

$$\mathcal{L}_{\text{dis}}^{-}(\mathcal{X}_U) = -\sum_{x_i \in \mathcal{X}_U} \sum_{c} \left| p_c^1(x_i) - p_c^2(x_i) \right|. \tag{6}$$

It can be seen that (5) contains two constraints. First, it is required that both classifiers correctly predict the classes of the training samples [the first two terms of (5)]. This ensures that the classifiers remain discriminative in the feature space. Second, in the last term of (5) [i.e., (6)], the classifiers' prediction discrepancy on the testing set is maximized, which constrains the classifiers to disagree with each other for testing samples. When (5) is minimized under these two constraints, the testing samples that are aligned with the training set will be consistently classified by the classifiers, while those that are not aligned with the training set will be discrepantly classified. Accordingly, the discrepancy of the classifiers' predictions can be used as a measure of alignment, and the testing samples with large discrepancies are then identified as biased samples (see Fig. 2).

2) Min-Step: Feature Distribution Bias Reduction: By maximizing the classifiers' prediction discrepancy, the biased samples can be identified in a fixed feature space. In the subsequent step, we fine-tune the parameters θ_g of the feature extractor in turn while holding the parameters θ_{f_1} and θ_{f_2} of the classifiers fixed [see Fig. 3(b) (right)]. The feature space is then adjusted to minimize the classifiers' prediction discrepancy. The loss function for the min-step is defined as

$$\mathcal{L}_{\text{Min}}(\mathcal{X}) = \mathcal{L}_{\text{cls}}^1(\mathcal{X}_L) + \mathcal{L}_{\text{cls}}^2(\mathcal{X}_L) + \lambda \mathcal{L}_{\text{dis}}^{+}(\mathcal{X}_U) \tag{7}$$

where

$$\mathcal{L}_{\text{dis}}^{+}(\mathcal{X}_U) = \sum_{x_i \in \mathcal{X}_U} \sum_{c} \left| p_c^1(x_i) - p_c^2(x_i) \right|. \tag{8}$$

When (7) is minimized, the two classifiers will agree with each other when generating predictions on both the training and testing sets. Because the parameters θ_{f^1} and θ_{f^2} are fixed, the classification boundary remains unchanged during the min-step. This encourages the feature extractor to push the identified features toward the classification boundary. Consequently, the training and testing feature distributions are aligned in consideration of their classes, which ensures that the classifiers are discriminative for the testing set (see Fig. 2).

During the training process, APL is implemented by iteratively training the PLM and AAM. Biased features are progressively identified by the classifiers and aligned with the training set by the feature extractor. The classifiers and the feature extractor play adversarial roles in attacking the distribution bias between the training and testing sets. At the same time, the testing set also participates in the feature learning procedure to allow robust representations to be obtained for the sample distribution.

D. Analysis of APL

1) Theoretical Analysis: In this subsection, we conduct a theoretical analysis based on the theory proposed in [72] to demonstrate how the proposed APL method lowers the upper error bound. Let $h, h' \in \mathcal{H}$ denote two hypotheses in the hypothesis space $\mathcal{H}$, where $h(x)$ classifies each sample x with a value of 1 (sample from the labeled set) or 0 (sample from the unlabeled set). The $\mathcal{H} \triangle \mathcal{H}$-distance is defined based on the symmetric difference hypothesis space as

$$\begin{aligned} &d_{\mathcal{H}\triangle\mathcal{H}}(\mathcal{X}_L, \mathcal{X}_U) \\ &= 2 \sup_{h,h'} \left| p_{x \in \mathcal{X}_L}(h(x) \neq h'(x)) - p_{x \in \mathcal{X}_U}(h(x) \neq h'(x)) \right| \\ &= 2 \sup_{h,h'} \left| \epsilon_{\mathcal{X}_L}(h, h') - \epsilon_{\mathcal{X}_U}(h, h') \right|. \end{aligned} \tag{9}$$

In (9), $\epsilon(h, h')$ denotes the expectation that h and h' disagree with each other. For the first term, it is often assumed that a model trained on the labeled set can correctly classify labeled samples, i.e., $\epsilon_{\mathcal{X}_L}(h, h')$ can be assumed to be very small and negligible. $d_{\mathcal{H}\triangle\mathcal{H}}(\mathcal{X}_L, \mathcal{X}_U)$ is, therefore, mainly determined by the second term. It can be seen that $\epsilon_{\mathcal{X}_U}(h, h')$ is positively related to the discrepancy loss $\mathcal{L}_{\text{dis}}^{+}(\mathcal{X}_U)$ defined in (8) if we replace the binary indicators h and h' with the discrepancy defined by f^1 and f^2. Equation (9) is then approximated as

$$\begin{aligned} d_{\mathcal{H}\triangle\mathcal{H}}(\mathcal{X}_L, \mathcal{X}_U) &= 2 \sup_{h,h'} \left| \epsilon_{\mathcal{X}_L}(h, h') - \epsilon_{\mathcal{X}_U}(h, h') \right| \\ &\approx 2 \sup_{h,h'} \left| \epsilon_{\mathcal{X}_U}(h, h') \right| \\ &\propto 2 \sup_{h,h'} \left| \mathcal{L}_{\text{dis}}^{+}(\mathcal{X}_U) \right|. \end{aligned} \tag{10}$$

Accordingly, it can be inferred that $d_{\mathcal{H}\triangle\mathcal{H}}(\mathcal{X}_L, \mathcal{X}_U)$ is positively related to (8). Based on the theorem proposed in [72], we have the following inequality:

$$\epsilon_{\mathcal{X}_U}(h) \leq \epsilon_{\mathcal{X}_L}(h) + \frac{1}{2} d_{\mathcal{H}\triangle\mathcal{H}}(\mathcal{X}_L \mathcal{X}_U) + \lambda \tag{11}$$

where $\epsilon_{\mathcal{X}_L}(h)$ is the error on the training set under hypothesis h and is often required to be minimized on the labeled set during model training, while λ is a constant if the dataset and model are given. Equation (11) indicates that minimizing $d_{\mathcal{H}\triangle\mathcal{H}}(\mathcal{X}_L, \mathcal{X}_U)$ can lower the upper error bound of $\epsilon_{\mathcal{X}_U}(h)$ (AAM Min-step). It is also indicated by (10) that we should first maximize $\mathcal{L}_{\text{dis}}(\mathcal{X}_U)$ to calculate the supremum of $d_{\mathcal{H}\triangle\mathcal{H}}(\mathcal{X}_L, \mathcal{X}_U)$ (AAM Max-step). Consequently, the AAM is able to minimize the upper error bound of $\epsilon_{\mathcal{X}_U}(h)$ through a learning process with the prototypical classifiers.

2) Effect of APL for Feature Extraction: In this subsection, we show that APL facilitates the extraction of long-range spatial feature dependencies. In Fig. 4(a) (taking the samples represented by green triangles as an example), before APL is performed, the HSI classification model extracts only short-range feature dependencies among HSI samples and ignores dependent samples that are far separated in HSI. This is caused by the local spatial receptive field of the convolution operation, as a cascade of multiple convolution operations still produces a limited spatial receptive field. APL identifies unlabeled samples that are far from labeled samples in HSI but close to labeled samples in the feature space [see Fig. 4(b)]. In this way, APL establishes long-range spatial feature dependencies among labeled and unlabeled samples. After APL is performed, the learned features are aware of both short-range and long-range spatial feature dependencies [see Fig. 4(c)], and their generalization ability is, therefore, improved.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

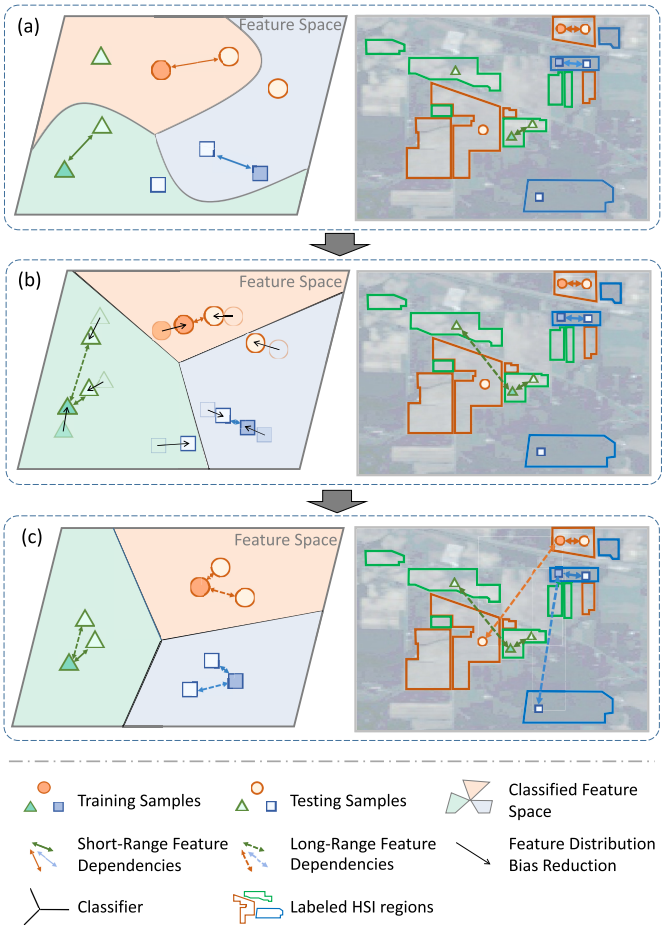


Fig. 4. Illustration of how APL facilitates the extraction of long-range feature dependencies. HSI samples are demonstrated in feature space and HSI (a) before performing APL, (b) during performing APL, and (c) after performing APL.

IV. Adversarial Prototypical Network

To implement the proposed APL in a DL framework, we propose the novel APNet architecture. As described in Section III, the proposed APNet consists of a feature extractor and two adversarial classifiers, as illustrated in Fig. 5.

A. Feature Extractor

As the key to HSI classification, we design a feature extractor that considers both spatial feature extraction and spectral feature extraction. As shown in Fig. 5, the feature extractor consists of four ResSE blocks and multiple convolutional layers for multiscale feature fusion. In the last three ResSE blocks, the size of the feature maps is downsampled by a factor of 2. In this way, the feature extractor outputs features with four scales, i.e., 1, 1/2, 1/4, and 1/8 of the input size. Each ResSE block is designed with multiple convolutional layers to extract spatial features at a specific scale and a squeeze-and-excitation operation [16] to extract spectral features. A group of convolutional layers and upsampling layers is then applied to fuse the multiscale features in a learnable way.

1) ResSE Block for Spatial and Spectral Feature Extraction: The ResSE block architecture is shown in Fig. 6. It is a combination of a residual operation [15] and a squeeze-and-excitation (SE) operation [16]. The first input layer is a 1×1 convolutional layer, which extracts the spectral features of each input pixel. The spectral features are input to a batch normalization (BN) layer for normalization, which is followed by an ReLU layer to activate the spectral features with high confidence. The activated features are then fed into a 3×3 convolutional layer, which is able to capture the spatial features in the 3×3 neighborhood of each input pixel while simultaneously considering spectral information. The output spatial–spectral features are then normalized by a BN layer and activated by an ReLU layer. An SE operation is used to introduce the spectral attention mechanism. The SE operation first converts the input feature map ($1 \times N \times W \times H$) into a feature vector ($1 \times N$) using a global average pooling (GAP) layer, where N, W, and H denote the number of spectral channels, the spatial width, and the spatial height, respectively, of the input feature map. The feature vector is then squeezed into a $1 \times N/16$ vector by a fully connected (FC1) layer and then excited back to a $1 \times N$ attention vector by another fully connected (FC2) layer. The attention vector is then multiplied in a channelwise manner (Multiply) with the input feature map of the SE operation to enhance useful spectral channels while suppressing noisy or redundant spectral channels. The feature map is further summed with the features' output by a shortcut connection. The shortcut connection consists of a $2\times$ downsampling ($2\times$ Down) layer and a 1×1 convolutional layer. Note that the $2\times$ Down layer exists only in ResSE blocks 2–4. The introduction of the shortcut connection allows the mainstream layers to learn the residuals between the input and output, which improves the capacity of the network to learn representative features without overfitting [15].

As described above, an ResSE block is able to learn both spectral and spatial features, with convolutional layers to learn spatial features, the SE operation to identify informative spectral features, and the residual operation to improve the learning capacity of the network. However, a single ResSE block can perceive the content of only a small spatial region for each pixel, i.e., a 3×3 region. On the other hand, a multiblock architecture with various feature scales has been widely adopted in many methods [73], [74]. Accordingly, we use four ResSE blocks to extract features, with downsampling layers in the last three blocks to enlarge the perception of the spatial contents. In this way, we can extract large feature maps that are sensitive to small-scale information, such as edges or textures, and small feature maps that are beneficial for the learning of large-scale information, such as semantic regions.

2) Multiscale Feature Fusion: Through forward propagation in the ResSE blocks, we obtain four feature maps of different scales. The next step is to classify the input features. However, two problems should be addressed.

1) The scales of the last three feature maps are different from that of the input HSI. This makes the relationships between samples and features unclear.
2) The scales and channel numbers of the four feature maps are different from each other. This requires the classifier to be capable of dealing with multiscale features.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

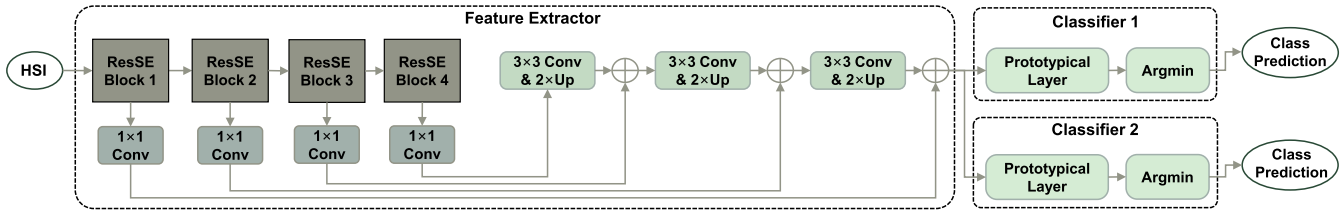


Fig. 5. Architecture of the proposed APNet.

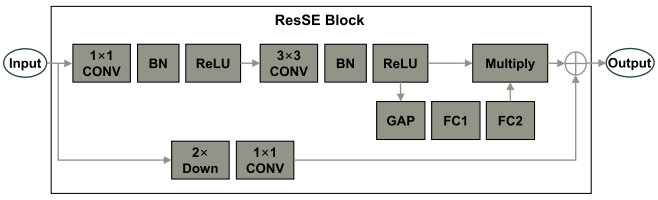


Fig. 6. Architecture of an ResSE block.

An intuitive approach is to rescale these feature maps to the size of the input HSI and classify the concatenated features. However, in this approach, the features at each scale are treated with equal importance. Inspired by the feature pyramid network (FPN) architecture [75], we propose a means of hierarchically merging the features in a learnable way. The architecture for feature fusion is shown in Fig. 5. The numbers of channels of the feature maps output by the ResSE blocks, from first to last, are 96, 128, 192, and 256. To merge these feature maps for classification, a 1×1 convolutional layer is first used beside each block to convert them to the same number of channels (128 in this article). The converted features are then hierarchically added and merged by a 3×3 convolutional layer, from ResSE block 4 to ResSE block 1. In this way, the features at each scale are progressively fused, and finally, features with spatial, spectral, and multiscale information are obtained, where each deep pixel (i.e., a feature vector corresponding to a specific location in the feature maps) is the feature of the correspondingly located HSI sample.

B. Sample Classifier

Two sample classifiers, which take the merged features as input, are added atop the feature extractor to assign each sample to a class. Each sample classifier consists of a prototypical layer that performs (1) and an Argmin layer that performs (3). For each sample classifier, prototypes are learned for each class by minimizing (2), and a metric space is established in which classification is performed by means of (3).

C. Training and Testing

During network training, the PLM and AAM are trained iteratively (see Fig. 3). First, we train the PLM to minimize the HSI classification loss defined in (2) by fine-tuning all parameters on the training set. Thus, the initial feature extractor and prototypical classifiers are obtained. Then, we train the AAM in an adversarial manner. We first fix the parameters of the ResSE blocks and only fine-tune those of the classifiers. The discrepancy losses defined in (6) and (8) are added to maximize the classifiers' prediction discrepancy. The classifiers are then updated to identify the biased feature distribution in the testing set. Subsequently, we fix the parameters of the classifiers and fine-tune the parameters of the ResSE blocks. Thus, the feature extractor is updated to minimize the classifiers' prediction discrepancy and align the identified features of the testing set. This alternating training of the PLM and AAM is repeated several times until the model is optimized and the feature distributions are aligned. During testing, we use the average prediction of the two classifiers for performance evaluation.

V. Experimental Results and Analysis

In this section, we first introduce the datasets used in the experiments. Detailed experimental settings are given. We then compare the proposed APNet with state-of-the-art methods. To demonstrate the effectiveness of the proposed method, we present an ablation study and a visualization analysis for APNet.

A. Datasets

1) University of Pavia: The University of Pavia dataset was acquired by the ROSIS sensor during a flight campaign over Pavia, Northern Italy. It has 103 spectral bands from 0.43 to 0.86 μm and a geometric resolution of 1.3 m. The image size is 610×340 pixels, of which 42 776 pixels are labeled with nine classes. The labels and the number of labeled samples are listed in Table I.

2) Indian Pines: The Indian Pines dataset was acquired by the AVIRIS sensor over the Indian Pines test site in Northwestern Indiana. It consists of 145×145 pixels and 224 spectral reflectance bands in the wavelength range of 0.4–2.5 μm, including agricultural areas, forest, and other natural perennial vegetation. A total of 10 249 pixels are labeled with 16 classes. The number of bands can be reduced to 200 by removing the bands covering regions of water absorption: [104–108], [150–163], and 220. The labels and the number of labeled samples are listed in Table I.

3) Salinas: This dataset, with 224 bands, was also collected by the AVIRIS sensor over Salinas Valley, California, USA. The spatial resolution is 3.7 m/pixel. The image size is 512×217 pixels. As with the Indian Pines dataset, we discarded the 20 water absorption bands. This image is available only as at-sensor radiance data. It contains 16 classes, including vegetation, bare soils, and vineyard fields. The labels and the number of labeled samples are listed in Table I.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LAND COVER CLASSES AND NUMBERS OF ANNOTATED SAMPLES IN THE UNIVERSITY OF PAVIA, INDIAN PINES, SALINAS, AND KENNEDY SPACE CENTER (KSC) DATASETS

University of Pavia			Indian Pines			Salinas			KSC		
Label	Class	Samples	Label	Class	Samples	Label	Class	Samples	Label	Class	Samples
1	Asphalt	6631	1	Alfalfa	46	1	Green_weeds1	2009	1	Scrub	761
2	Meadows	18649	2	Corn-notill	1428	2	Green_weeds2	3726	2	W. swamp	243
3	Gravel	2099	3	Corn-mintill	830	3	Fallow	1976	3	CP/Oak	256
4	Trees	3064	4	Corn	237	4	Fallow_plow	1394	4	Slash pine	252
5	Metal sheets	1345	5	Grass-pasture	483	5	Fallow_smooth	2678	5	Oak/B.leaf	161
6	Bare soil	5029	6	Grass-trees	730	6	Stubble	3959	6	Hardwood	229
7	Bitumen	1330	7	Grass-pasture.M	28	7	Celery	3579	7	Swamp	105
8	Bricks	3682	8	Hay-windrowed	478	8	Grapes_UT	11271	8	Gram. marsh	431
9	Shadows	947	9	Oats	20	9	Soil_vineyard	6203	9	Spart. marsh	520
			10	Soybean-notill	972	10	Senesced_weeds	3278	10	Catt. marsh	404
			11	Soybean-mintill	2455	11	Romaine_4wk	1068	11	Salt marsh	419
			12	Soybean-clean	593	12	Romaine_5wk	1927	12	Mud flats	503
			13	Wheat	205	13	Romaine_6wk	916	13	Water	927
			14	Woods	1265	14	Romaine_7wk	1070			
			15	Buildings-G.T.D	386	15	Vineyard_UT	7268			
			16	Stone-Steel.T	93	16	Vineyard_trellis	1807			
Total		42776	Total		10249	Total		54129	Total		5211

4) Kennedy Space Center (KSC): The KSC dataset, with 224 bands, was also collected by the AVIRIS sensor over the Kennedy Space Center (KSC), Florida, USA. The KSC data, acquired from an altitude of approximately 20 km, have a spatial resolution of 18 m. After removing water absorption bands and bands with a low signal-to-noise ratio (SNR), 176 bands were used for the analysis. Discrimination of land cover for this environment is difficult due to the similarity of the spectral signatures for certain vegetation types. Thirteen classes of various land cover types are defined for the site. The labels and the number of labeled samples are listed in Table I.

B. Experimental Details

1) Training Settings: In each experiment, we randomly selected K training pixels (samples) from the HSI for each class and tested the performance on the remaining HSI pixels. Each experiment was repeated ten times, and we report the average performance, including the overall accuracy (OA), the average accuracy (AA), and the kappa coefficient (Kappa). The experiments were conducted using the stochastic gradient descent (SGD) optimizer with an initial learning rate of 0.2 and a weight decay of 0.0001. For APL, we repeated PLM and AAM training five times, with 100 forward and backward iterations in the initial step and 50 iterations in each remaining step. The learning rate was set to 0.02 after PLM, and AAM training was repeated three times. The hyperparameter λ defined in (5) and (7) was empirically set to 0.5.

2) Methods for Comparison: We compared the proposed APNet with five state-of-the-art DL methods, namely, 3-D-CNN [22], pResNet [23], ASSMN [31], CBW [32], and FreeNet [24], as well as the conventional SVM method [12].

1) *SVM [12]:* The HSI pixels are directly fed into the SVM classifier. We used grid search to find the best hyperparameters.
2) *3-D-CNN [22]:* In 3-D-CNN, an HSI patch is regarded as a training sample for the center pixel. The network contains four 3-D convolutional layers, two 3-D pooling layers, and a linear layer for classification. During training, the batch size was set to min(K, 100). The learning rate was set to 0.01, and the weight decay was set to 0.0005. The number of training epochs was set to 1000.
3) *pResNet [23]:* The pyramidal residual network (pResNet) architecture is composed of convolutional layers and three pyramidal residual modules. The size of an HSI patch was set to 11, and the batch size was set to min(K, 100). The model was trained for 200 epochs with a learning rate of 0.1. The hyperparameter α of pResNet was set to 100.
4) *Adaptive Spectral–Spatial Multiscale Network (ASSMN) [31]:* The ASSMN architecture consists of individual subnetworks for spectral feature extraction and spatial feature extraction. The features extracted by the subnetworks are fused via an adaptive feature combination strategy. In our experiments, we used the official code of ASSMN with the default settings.
5) *Compact Band Weighting (CBW) [32]:* A single 2D-CNN is used as the backbone network for HSI classification. The Adam optimizer and a cross-entropy loss are used during training. A plug-and-play CBW module based on an attention-driven mechanism is added to the backbone for HSI classification. In our experiments, we used the official code of CBW with the default settings.
6) *FreeNet [24]:* The HSI image is directly input into FreeNet, and a prediction map of the same spatial size as the HSI image is output. FreeNet is composed of an encoder network and a decoder network with spectral attention modules. In our experiments,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

TABLE II

COMPARISON OF THE PER-CLASS ACCURACY, OA, AA, AND KAPPA OF APNET WITH THOSE OF CONVENTIONAL AND STATE-OF-THE-ART METHODS ON THE UNIVERSITY OF PAVIA DATASET WITH DIFFERENT NUMBERS OF TRAINING SAMPLES PER CLASS (K)

Per-Class Acc.	SVM [12]			3D-CNN [22]			pResNet [23]			ASSMN [31]			CBW [32]			FreeNet [24]			APNet		
	K			K			K			K			K			K			K		
	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5
1	46.6	56.3	57.9	19.5	44.1	60.2	40.6	49.4	59.7	27.6	38.8	53.7	43.2	43.5	56.2	48.7	52.3	75.9	**63.9**	**72.7**	**92.3**
2	27.6	47.6	54.0	52.7	**62.1**	62.4	47.3	44.9	57.8	50.6	59.3	69.3	**61.9**	57.8	**72.5**	53.0	59.9	68.7	41.8	49.6	68.4
3	36.3	37.6	53.0	42.0	55.9	67.1	30.1	44.6	60.8	18.8	25.8	40.6	45.0	43.2	60.3	29.6	**58.5**	57.7	**51.7**	58.4	**89.3**
4	76.9	78.7	78.9	57.2	82.0	81.2	57.3	71.3	83.5	22.5	42.0	66.4	44.9	79.4	85.5	62.2	87.7	95.2	**91.6**	**94.4**	**97.4**
5	75.0	75.9	96.2	66.8	94.4	93.9	78.4	85.3	92.7	50.3	75.0	86.0	90.0	96.8	98.5	95.2	92.6	99.5	**98.1**	**100**	**100**
6	42.4	37.0	65.2	37.2	41.1	48.3	23.3	51.0	58.5	48.2	37.2	59.7	26.6	58.5	68.8	**58.9**	56.9	**92.4**	53.4	**70.6**	91.5
7	74.3	84.0	87.1	**92.2**	86.6	85.2	55.7	62.1	74.7	61.8	51.2	72.9	53.6	67.4	78.7	73.8	89.0	84.1	49.0	**93.8**	**99.9**
8	54.2	55.5	60.4	45.2	55.9	70.8	50.8	44.4	70.2	45.0	53.5	67.4	23.1	34.6	50.8	36.5	58.7	80.6	**56.2**	**89.6**	**95.7**
9	**99.9**	**99.9**	99.8	95.1	95.7	96.6	71.4	83.4	93.7	66.0	80.3	90.3	74.9	81.4	95.1	86.2	94.0	99.1	99.4	99.8	**99.9**
OA	43.1	53.3	61.6	46.1	58.2	63.4	45.4	50.8	63.6	43.4	50.8	65.1	50.4	56.5	69.5	53.8	62.8	77.1	**55.2**	**66.8**	**82.9**
AA	59.2	63.6	72.5	56.4	68.7	74.0	50.5	59.6	72.4	43.4	51.4	67.4	51.5	62.5	74.1	60.5	72.2	83.7	**67.3**	**81.0**	**92.7**
Kappa	33.8	43.5	53.0	36.2	50.4	56.8	33.9	40.8	55.0	30.9	38.3	55.7	36.3	46.6	61.5	44.0	54.1	71.4	**46.6**	**60.0**	**78.7**

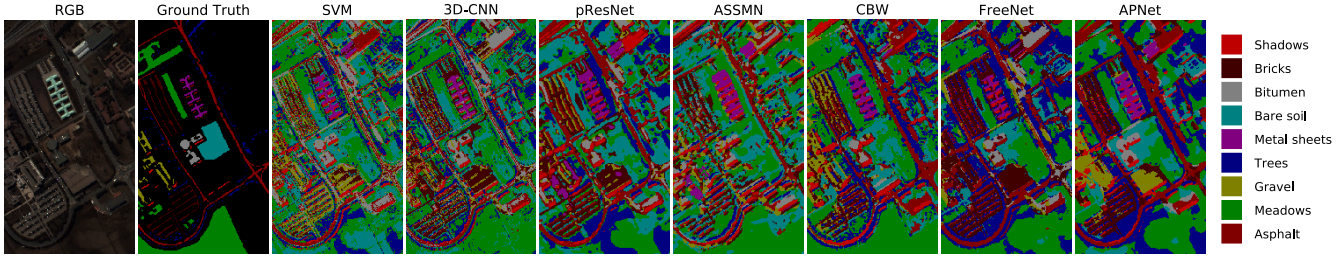


Fig. 7. Prediction results of APNet and the methods considered for comparison on the University of Pavia dataset.

we used the official code of FreeNet with the default settings.

7) *Baseline:* For further comparison, we removed one classifier from APNet to construct a baseline network in which ResSE modules are used to extract features, and a single 1×1 2-D convolutional layer followed by a cross-entropy loss is used to train an HSI classifier. The learning rate was set to 0.01 in the first 500 iterations and 0.001 in the remaining 250 iterations.

C. Performance Comparison

In the following subsections, we compare the proposed APNet with the state-of-the-art methods listed above on the four publicly available datasets introduced previously.

1) University of Pavia: Table II shows the performance of APNet and the methods considered for comparison with small numbers of training samples, i.e., $K \in \{1, 2, 5\}$, on the University of Pavia dataset. When $K = 1$, APNet significantly outperforms the state-of-the-art FreeNet method by 1.4% (from 53.8% to 55.2%), 6.8% (from 60.5% to 67.3%), and 2.6% (from 44.0% to 46.6%) in terms of the OA, AA, and Kappa metrics, respectively. When K is increased to 5, APNet continues to outperform FreeNet, by 5.8% (from 77.1% to 82.9%), 9.0% (from 83.7% to 92.7%), and 7.3% (from 71.4% to 78.7%) in terms of OA, AA, and Kappa, respectively. These performance gains clearly show that APNet is superior to the state-of-the-art methods due to its use of APL to alleviate overfitting and align the feature distributions. It is also noteworthy that, with very small training samples ($K \in \{1, 2, 5\}$), the DL methods 3-D-CNN and pResNet achieve worse performance than the conventional SVM method. This indicates that 3-D-CNN and pResNet have a poor ability to generalize a model trained on a small training set to a testing set. In contrast, APNet utilizes prototypical classifiers to align the feature distributions of the training set and testing set and, therefore, improves the generalization ability of a model trained on a small set.

Fig. 7 shows the prediction maps generated by APNet and the compared methods with $K = 5$ on the University of Pavia dataset. The prediction map of the proposed APNet method is closest to the ground-truth map. The prediction map of APNet contains fewer noisy predictions than those of SVM, 3-D-CNN, pResNet, ASSMN, CBW, and FreeNet, especially for shadows. The reason is that shadow regions tend to contain less informative content or noisy backgrounds and, therefore, can give rise to a large distribution bias between the training and testing sets. Models trained on such a small and biased set cannot be readily generalized to the testing set. It is also shown in Fig. 7 that the DL method 3-D-CNN performs even worse than the SVM method for shadows. In contrast, the proposed APNet method is able to alleviate overfitting and reduce the bias and, therefore, achieves a more accurate prediction map.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

TABLE III

COMPARISON OF THE PER-CLASS ACCURACY, OA, AA, AND KAPPA OF APNET WITH THOSE OF CONVENTIONAL AND STATE-OF-THE-ART METHODS ON THE INDIAN PINES DATASET WITH DIFFERENT NUMBERS OF TRAINING SAMPLES PER CLASS (K)

Per-Class Acc.	SVM [12]			3D-CNN [22]			pResNet [23]			ASSMN [31]			CBW [32]			FreeNet [24]			APNet		
	K			K			K			K			K			K			K		
	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5
1	73.8	75.0	80.0	49.6	60.2	80.7	62.0	86.4	92.4	84.0	93.6	**99.5**	68.5	82.8	92.4	71.6	95.2	97.3	**96.9**	**98.0**	99.0
2	18.3	19.4	23.0	19.9	23.7	34.7	20.0	33.1	46.6	10.8	20.9	36.7	20.8	34.6	40.3	22.1	30.6	51.8	**27.0**	**36.4**	**58.3**
3	16.9	22.3	37.3	18.4	29.5	41.3	27.0	37.6	45.3	19.5	26.5	46.5	16.9	21.7	34.7	18.5	36.8	50.7	**30.0**	**43.6**	**72.1**
4	22.7	22.9	50.7	23.3	32.1	53.6	20.7	37.1	57.6	32.8	55.1	78.5	30.0	37.9	71.3	26.9	50.7	85.8	**43.6**	**72.0**	**93.3**
5	36.8	35.5	56.2	23.0	39.7	49.2	38.8	43.0	64.9	27.7	45.3	50.6	39.2	58.9	62.0	32.3	50.5	71.9	**55.4**	**73.0**	**79.6**
6	43.8	35.1	61.3	49.1	51.6	72.1	39.8	63.7	80.0	22.0	33.9	62.6	46.4	72.3	87.9	58.8	69.4	86.9	**61.0**	**79.1**	**95.9**
7	76.3	91.9	90.4	45.9	67.3	91.3	98.5	91.2	96.5	87.4	94.6	99.6	95.0	98.2	98.9	82.3	96.9	**100**	**100**	**100**	99.6
8	41.0	41.7	61.8	61.1	78.5	76.8	56.8	60.8	86.7	**81.8**	86.7	98.6	71.3	85.1	97.7	53.8	86.2	97.8	72.8	**96.8**	**100**
9	56.3	73.9	80.7	77.9	85.0	98.0	81.1	98.9	99.3	72.1	83.9	100	87.0	87.5	98.0	**99.4**	99.4	100	98.4	**100**	**100**
10	24.5	39.2	41.3	42.5	32.5	54.0	26.1	24.6	57.2	22.2	26.3	53.0	29.8	41.6	61.5	40.0	45.4	**70.4**	**49.8**	**56.7**	69.2
11	33.6	28.1	40.9	23.9	37.6	35.0	29.8	32.8	49.7	31.6	41.1	56.0	34.0	41.0	48.1	33.4	58.7	65.7	**45.0**	**63.5**	**68.0**
12	20.4	14.5	30.0	18.3	23.7	37.4	14.9	18.7	31.8	24.9	29.4	41.9	12.7	30.0	29.1	23.7	31.3	53.1	**28.3**	**44.0**	**63.0**
13	85.1	92.8	90.8	49.5	70.6	90.3	58.1	76.5	96.0	57.4	72.2	95.0	74.2	79.7	98.4	74.6	88.9	97.4	**99.6**	**99.4**	**100**
14	46.4	58.3	68.1	56.2	57.9	75.0	46.1	63.9	73.7	59.7	47.4	79.3	60.0	60.5	83.7	**63.0**	75.1	86.4	59.4	**79.9**	**87.4**
15	23.2	15.4	29.2	21.3	18.1	40.0	28.8	37.2	57.5	25.0	58.0	68.2	24.9	47.4	66.1	32.6	61.4	88.8	**46.5**	**66.0**	**90.3**
16	68.8	66.9	89.0	59.5	83.7	85.0	79.5	93.4	91.4	62.3	78.0	95.2	90.3	90.2	97.3	86.9	96.3	97.8	**99.2**	**99.8**	**98.3**
OA	32.3	33.4	45.5	32.4	39.2	49.5	32.6	41.6	58.3	32.4	39.9	58.9	36.5	47.3	59.4	38.6	55.0	70.6	**47.5**	**63.1**	**75.8**
AA	43.0	45.8	58.2	40.0	49.5	63.4	45.5	56.2	70.4	45.1	55.8	72.6	50.1	60.6	73.0	51.2	67.1	81.4	**63.3**	**75.5**	**85.9**
Kappa	25.0	26.6	39.2	25.7	32.8	44.6	25.5	35.3	53.1	26.2	33.6	53.9	29.4	41.1	54.5	31.5	49.2	66.8	**41.4**	**58.5**	**72.7**

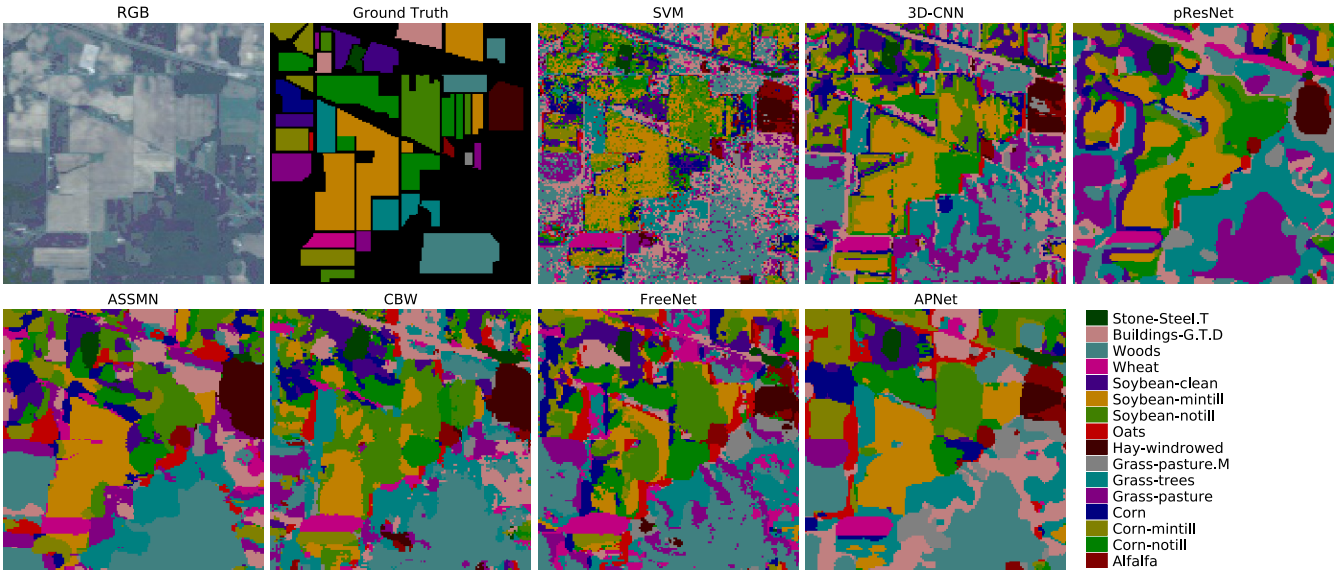


Fig. 8. Prediction results of APNet and the methods considered for comparison on the Indian pines dataset.

2) Indian Pines: Table III shows the performance of APNet and the compared methods on the Indian Pines dataset. The proposed APNet method again achieves the best OA, AA, and Kappa among the compared methods with all values of K. In this case, APNet significantly outperforms the state-of-the-art FreeNet method by 8.9% (from 38.6% to 47.5%), 12.1% (from 51.2% to 63.3%), and 9.9% (from 31.5% to 41.4%) in terms of the OA, AA, and Kappa metrics, respectively, when $K = 1$, and by 5.2% (from 70.6% to 75.8%), 4.5% (from 81.4% to 85.9%), and 5.9% (from 66.8% to 72.7%), respectively, when $K = 5$. These significant performance gains clearly demonstrate the effectiveness of prototypical classifiers and feature distribution alignment for improving model generalization ability. Similar to the observations on the University of Pavia dataset, the DL method 3-D-CNN achieves comparable or worse performance relative to the conventional SVM method when $K = 1$. APNet again shows its superiority in allowing generalizable models to be learned from a small training set.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

COMPARISON OF THE PER-CLASS ACCURACY, OA, AA, AND KAPPA OF APNET WITH THOSE OF CONVENTIONAL AND STATE-OF-THE-ART METHODS ON THE SALINAS DATASET WITH DIFFERENT NUMBERS OF TRAINING SAMPLES PER CLASS (K)

Per-Class Acc.	SVM [12]			3D-CNN [22]			pResNet [23]			ASSMN [31]			CBW [32]			FreeNet [24]			APNet		
	K			K			K			K			K			K			K		
	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5
1	94.7	94.8	95.4	89.1	93.4	99.3	81.9	71.5	84.6	65.9	64.8	86.3	96.4	68.6	91.5	72.2	86.9	98.6	**99.5**	**99.9**	**100**
2	76.1	66.5	94.2	86.0	97.2	91.9	85.9	90.1	97.6	62.1	60.4	79.5	59.2	94.4	92.8	84.2	92.7	94.3	**96.0**	**97.6**	**100**
3	40.1	44.3	82.6	65.4	74.9	79.8	57.8	54.1	84.4	54.1	69.3	89.4	73.0	52.6	75.4	**77.9**	**88.6**	**95.0**	67.4	84.1	94.5
4	96.2	96.1	98.2	92.1	95.8	97.3	69.5	86.8	93.5	86.0	95.5	99.4	96.9	98.5	97.2	88.0	99.0	99.6	**99.7**	**99.9**	**99.8**
5	**97.7**	**95.7**	94.1	66.0	88.3	83.6	59.0	86.2	87.2	66.2	79.1	**98.2**	39.2	80.4	86.2	67.5	86.6	94.0	91.0	93.4	97.6
6	95.3	96.2	98.2	88.6	98.8	98.5	83.9	94.3	97.5	80.0	90.0	97.7	97.6	98.1	99.7	99.5	99.4	99.1	**99.9**	**100**	**99.7**
7	89.2	93.3	95.5	67.4	96.0	97.9	84.5	96.9	97.8	75.4	85.5	93.2	80.4	94.7	96.9	97.9	98.9	99.5	**99.9**	**99.9**	**100**
8	44.6	37.1	55.1	**53.7**	44.1	65.3	33.4	62.2	61.9	43.4	54.0	46.5	48.4	55.7	**62.2**	44.0	40.8	70.0	51.9	55.9	**86.2**
9	77.6	94.2	97.4	97.6	98.3	98.8	72.4	95.0	98.2	65.0	88.1	97.2	85.9	96.9	97.9	96.4	99.6	99.8	**99.6**	**99.8**	**99.9**
10	33.5	31.0	67.5	32.6	62.0	80.4	33.2	46.8	80.0	37.8	59.2	90.7	47.6	64.2	91.0	46.8	69.7	90.1	**61.2**	**86.2**	**92.7**
11	53.3	77.5	87.9	56.6	82.0	94.4	61.3	57.4	93.9	61.9	77.8	93.0	52.0	57.7	93.4	72.8	92.5	96.5	**88.3**	**96.9**	**98.7**
12	69.9	89.4	97.7	65.3	93.5	96.3	76.5	82.4	95.1	68.3	85.8	90.6	81.5	82.4	97.0	83.0	90.4	99.1	**96.4**	**99.4**	**100**
13	98.7	94.3	97.3	57.1	90.3	98.9	98.2	95.5	99.0	95.6	93.2	99.0	94.2	96.7	99.8	88.9	**99.7**	**100**	**99.8**	98.5	99.2
14	86.8	86.5	88.7	71.4	81.7	92.8	92.9	97.3	97.6	69.4	85.2	95.5	79.8	94.5	98.7	92.6	**99.4**	**99.8**	**95.3**	95.4	97.7
15	52.6	67.3	59.1	48.5	64.6	45.1	**75.2**	70.0	78.6	62.5	68.0	72.6	70.2	81.6	72.4	68.3	**87.5**	**94.3**	69.5	87.1	90.5
16	53.0	41.0	90.0	59.0	59.0	84.4	63.0	76.2	85.4	33.6	68.2	78.0	64.4	80.7	**96.5**	64.8	78.6	94.1	**80.8**	**93.1**	95.2
OA	66.4	68.9	80.0	66.0	75.2	79.6	64.4	76.9	84.5	60.0	71.9	79.9	68.7	78.7	84.6	72.8	80.7	91.0	**80.4**	**86.7**	**94.8**
AA	72.5	75.3	87.4	68.5	82.5	87.8	70.5	78.9	89.5	64.2	76.5	87.9	72.9	81.1	90.6	77.8	88.1	95.2	**87.3**	**92.9**	**97.0**
Kappa	62.8	65.7	77.8	63.3	73.8	78.6	60.9	74.4	82.8	55.9	68.9	77.8	65.4	76.4	82.9	69.9	78.7	90.1	**78.3**	**85.3**	**94.2**

TABLE V

COMPARISON OF THE PER-CLASS ACCURACY, OA, AA, AND KAPPA OF APNET WITH THOSE OF CONVENTIONAL AND STATE-OF-THE-ART METHODS ON THE KSC DATASET WITH DIFFERENT NUMBERS OF TRAINING SAMPLES PER CLASS (K)

Per-Class Acc.	SVM [12]			3D-CNN [22]			pResNet [23]			ASSMN [31]			CBW [32]			FreeNet [24]			APNet		
	K			K			K			K			K			K			K		
	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5	1	2	5
1	34.8	53.3	82.2	66.6	77.4	75.8	77.7	90.6	96.9	56.1	55.5	66.4	80.4	64.7	94.2	**93.5**	**98.8**	98.0	92.6	96.4	**98.6**
2	39.4	35.3	72.4	41.5	63.5	71.6	14.8	37.0	80.3	30.3	42.6	73.4	10.0	16.0	30.0	35.0	57.3	80.9	**55.9**	**84.6**	**96.3**
3	33.9	47.8	70.4	48.7	33.0	44.7	42.6	61.3	78.1	42.0	63.0	69.1	20.9	38.3	50.0	67.1	74.6	95.2	**90.8**	**96.2**	**99.8**
4	24.5	29.0	33.9	17.7	21.3	21.9	25.7	27.2	37.1	19.8	28.1	51.7	12.9	21.0	34.1	21.0	**58.0**	71.9	**30.4**	50.9	**76.1**
5	25.2	16.9	48.3	17.7	43.8	39.9	39.1	45.3	57.8	45.8	71.6	59.6	23.4	28.4	55.9	46.0	58.3	84.7	**67.4**	**82.7**	**90.7**
6	36.0	37.9	50.2	26.7	39.0	49.8	33.6	38.4	85.8	40.8	61.8	44.0	20.3	44.3	34.1	61.6	84.6	93.5	**63.9**	**87.5**	**99.9**
7	50.6	36.0	84.9	63.7	67.7	71.5	71.6	88.5	92.5	68.8	74.1	93.2	20.5	64.3	96.2	97.4	99.4	100	**98.8**	**100**	**100**
8	19.8	30.4	65.9	31.3	32.6	48.6	34.0	44.4	67.3	30.6	49.9	50.8	10.7	49.3	36.9	32.1	**67.8**	88.2	**47.4**	64.1	**99.0**
9	36.4	41.0	75.6	42.7	64.2	69.8	56.9	**89.5**	88.3	88.8	81.4	78.5	**96.1**	85.4	**100**	76.6	77.1	87.0	79.9	83.7	96.7
10	32.1	29.3	66.2	44.2	41.0	70.8	48.0	44.0	86.4	32.9	66.8	78.4	9.7	17.0	46.5	60.7	85.4	91.9	**81.0**	**95.7**	**100**
11	82.0	82.4	87.8	66.8	94.8	91.9	89.0	90.6	94.0	47.9	67.5	89.8	87.6	75.3	98.3	82.5	93.9	**99.7**	**96.0**	**97.5**	98.4
12	54.0	65.0	78.7	50.4	69.3	72.7	41.8	30.7	87.9	22.5	38.5	82.4	17.6	19.8	24.9	24.1	65.9	**97.3**	**69.4**	**91.0**	96.6
13	99.9	99.9	99.8	98.9	99.5	100	82.1	87.2	100	100	100	100	71.2	75.5	99.1	100	100	100	**100**	**100**	**100**
OA	50.5	55.7	76.5	55.9	65.5	71.1	57.5	65.9	85.9	55.0	65.3	75.8	48.5	52.5	69.0	68.1	83.2	93.5	**79.3**	**89.1**	**97.3**
AA	43.8	46.5	70.5	47.4	57.5	63.8	50.5	59.6	80.9	48.2	61.6	72.1	37.0	46.1	61.6	61.3	78.5	91.4	**74.9**	**86.9**	**96.3**
Kappa	45.3	50.9	73.9	51.2	61.9	68.2	52.9	62.1	84.3	49.6	61.3	73.1	42.5	47.2	65.5	64.0	81.1	92.7	**77.0**	**87.9**	**97.0**

Fig. 8 shows the prediction maps of APNet and the compared methods with $K = 5$ on the Indian Pines dataset. APNet achieves the best visual results. Compared with the state-of-the-art methods, APNet avoids "oversegmenting" the prediction map, i.e., reduces the number of noisily predicted regions in Fig. 8. By aligning the identified feature distribution, APNet is able to reduce the risk of misclassifying a whole region.

3) Salinas: Table IV shows the performance of APNet and the compared methods on the Salinas dataset. The proposed APNet again achieves the best OA, AA, and Kappa among the compared methods. APNet significantly outperforms FreeNet by 7.6% (from 72.8% to 80.4%), 9.5% (from 77.8% to 87.3%), and 8.4% (from 69.9% to 78.3%) in terms of the OA, AA, and Kappa metrics, respectively, when $K = 1$, and by 3.8% (from 91.0% to 94.8%), 1.8% (from 95.2% to 97.0%),

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

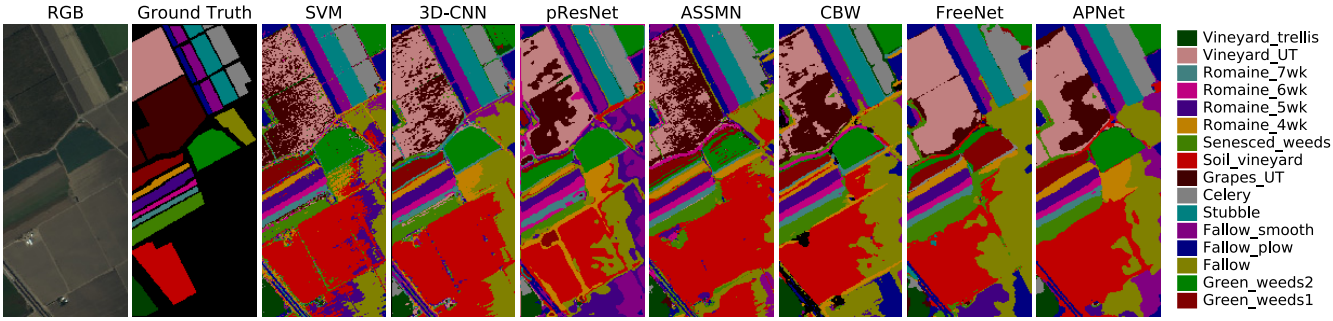


Fig. 9. Prediction results of APNet and the methods considered for comparison on the Salinas dataset.

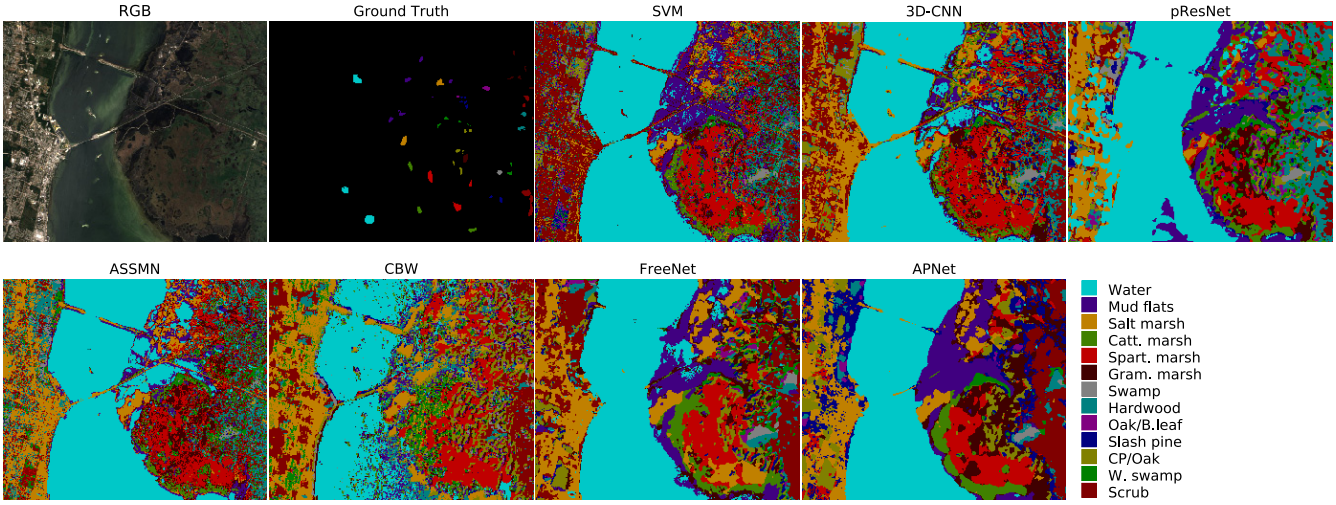


Fig. 10. Prediction results of APNet and the methods considered for comparison on the KSC dataset.

and 4.1% (from 90.1% to 94.2%), respectively, when $K = 5$. Different from the results on the University of Pavia and Indian Pines datasets, when $K = 5$, APNet only slightly outperforms FreeNet in the AA metric, by 1.8%. The reason is that the patterns in the Salinas dataset are simpler than those in the former two datasets. With only five samples per class, the AA performance already reaches 97%, which is relatively high, indicating that the bias between the training and testing sets rapidly becomes small. APNet is able to alleviate the overfitting problem and, therefore, offers improved performance.

Fig. 9 shows the prediction maps of APNet and the compared methods with $K = 5$ on the Salinas dataset. The prediction map of APNet is very close to the ground-truth map for most classes. Compared with the state-of-the-art methods, APNet reduces the risk of misclassifying pixels as similar classes, e.g., misclassifying "Grapes_UT" as "Vineyard_UT."

4) Kennedy Space Center (KSC): Table V shows the performance of APNet and the compared methods on the KSC dataset. Consistent with the results on the previous datasets, the proposed APNet method achieves the best OA, AA, and Kappa among the compared methods with all values of K. APNet outperforms FreeNet by 11.2% (from 68.1% to 79.3%), 13.6% (from 61.3% to 74.9%), and 13.0% (from 64.0% to 77.0%) in terms of the OA, AA, and Kappa metrics, respectively, when $K = 1$, and by 3.8% (from 93.5% to 97.3%), 4.9% (from 91.4% to 96.3%), and 4.3% (from 92.7% to 97.0%), respectively, when $K = 5$. Similar to the results on the Salinas dataset, the performance improvements are positively correlated with K. When $K \in \{1, 2, 5\}$, i.e., the bias between the training and testing sets and the risk of overfitting are large, APNet is able to reduce the bias and, therefore, significantly improve the performance. When $K = 5$, the bias and overfitting risk between the training and testing sets become small, and APNet still outperforms the state-of-the-art methods.

Fig. 10 shows the prediction maps of APNet and the compared methods with $K = 5$ on the KSC dataset. APNet achieves the best visual results among the compared methods. These results again demonstrate that, by aligning the identified feature distribution with prototypical classifiers, APNet is able to reduce the risk of overfitting and, therefore, improves the prediction results.

D. Ablation Analysis

1) Effect of the Number of Training Samples per Class (K): We trained APNet and the methods considered for comparison with $K \in \{1, 2, 5, 10, 20, 50, 100\}$ on the four datasets to analyze the effect of K. The results are shown in Fig. 11, from which it can be seen that APNet consistently outperformed the state-of-the-art models in all experiments. Even when K reaches 100, APNet still outperforms the state-of-the-art

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

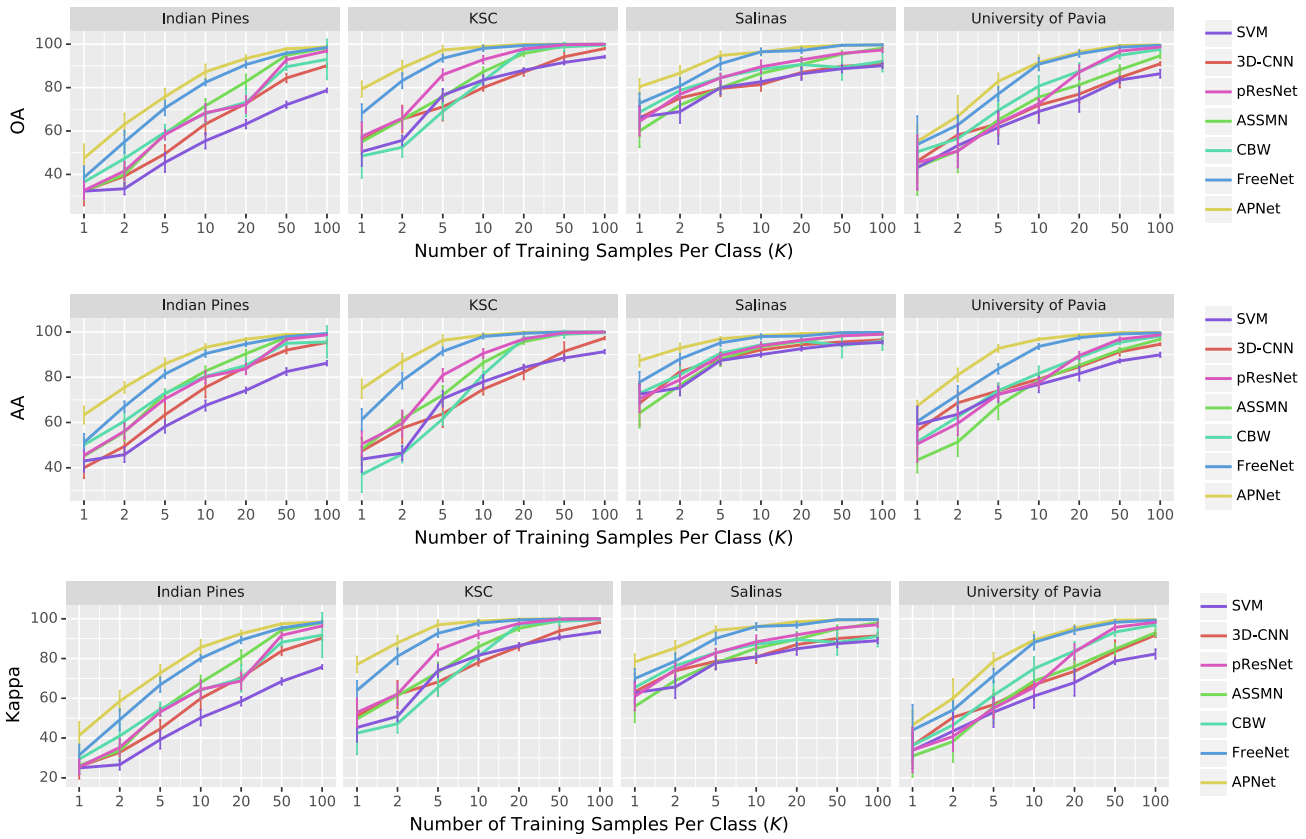


Fig. 11. Performance comparison with respect to the number of training samples per class (K).

methods. This indicates that APNet can also be applied when the training samples are large, showing its ability to generalize to various challenging settings.

2) Effect of the Proportion of Training Samples: We trained APNet and the methods considered for comparison with different proportions (0.5% and 5%) of training samples on the Indian Pines dataset, which suffers from the worst class imbalance problem among the four datasets. The results are shown in Table VI, from which it can be seen that APNet outperformed the state-of-the-art methods for almost all classes. Compared with the DL methods, pResNet, ASSMN, CBW, and FreeNet, APNet is more robust against class imbalance.

3) Effects of the PLM and AAM: To analyze the effects of the PLM and AAM, we compared the performance of the baseline method, the baseline method combined with the PLM, and the full proposed APNet architecture, which are denoted by "Baseline," "PLM," and "APNet" in Fig. 12. The experiments were conducted on the Indian Pines dataset with $K \in \{1, 2, 5, 10, 20, 50, 100\}$, and the results are shown in Fig. 12. It can be seen that, in comparison with the baseline method, the introduction of the PLM significantly improves the performance when $K \in \{1, 2, 5, 10, 20\}$. When $K > 20$, the architecture including the PLM also slightly outperforms or performs comparably to the baseline. These results validate that the introduction of the PLM improves the generalization ability of the model and, therefore, allows better performance to be achieved when the training set is small. The introduction

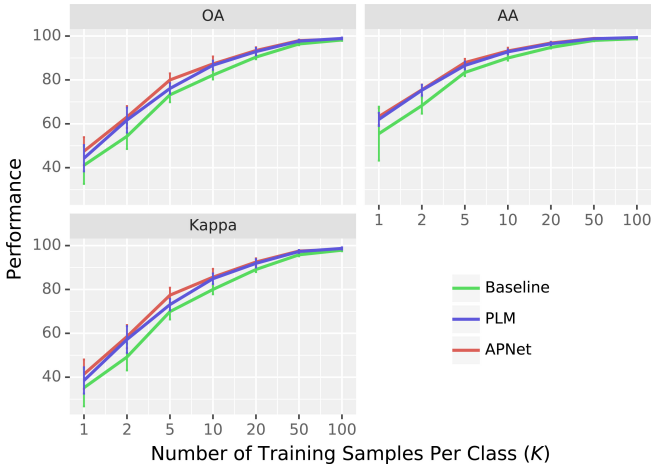


Fig. 12. Performance comparison of the baseline method ("Baseline"), baseline + PLM ("PLM"), and APNet.

of the AAM further improves the performance, showing the effectiveness of feature distribution alignment.

4) Visualization of Feature Alignment: To demonstrate the ability of APNet to align the feature distributions of the training and testing sets, we used the trained model to extract the features of the training and testing samples and visualized them using t-distributed stochastic neighbor embedding (t-SNE) [76]. For comparison, we visualized the features extracted by the baseline + PLM architecture and APNet,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

COMPARISON OF THE PER-CLASS ACCURACY, OA, AA, AND KAPPA OF APNET WITH THOSE OF CONVENTIONAL AND STATE-OF-THE-ART METHODS ON THE INDIAN PINES DATASET WITH DIFFERENT PERCENTAGES OF TRAINING SAMPLES

Per-Class	SVM [12]		3D-CNN [22]		pResNet [23]		ASSMN [31]		CBW [32]		FreeNet [24]		APNet	
Acc.	0.5%	5.0%	0.5%	5.0%	0.5%	5.0%	0.5%	5.0%	0.5%	5.0%	0.5%	5.0%	0.5%	5.0%
1	38.9	22.7	25.3	45.7	48.9	86.0	78.2	**95.5**	61.3	67.0	63.6	28.0	**88.0**	91.9
2	36.5	68.3	38.5	84.3	46.4	91.0	43.7	91.9	48.0	77.6	52.8	97.0	**68.2**	**97.5**
3	13.2	53.9	29.3	76.7	29.4	96.6	40.8	92.9	31.2	75.3	34.9	95.1	**62.9**	**98.3**
4	5.5	46.1	11.9	74.9	23.4	70.2	18.4	89.5	25.4	75.7	9.4	70.0	**74.6**	**96.6**
5	18.8	84.1	33.0	91.0	42.1	93.8	35.2	90.7	54.4	91.7	36.5	90.7	**66.0**	**95.5**
6	79.6	94.0	70.7	97.9	78.3	99.0	62.9	97.6	89.0	98.0	87.0	97.9	**95.1**	**99.2**
7	35.2	41.9	42.2	28.9	79.3	74.6	91.9	70.7	90.4	60.0	79.3	51.3	**94.4**	**89.9**
8	64.7	97.1	81.3	98.9	82.7	99.9	83.7	100	92.6	99.9	88.2	98.6	**98.0**	**100**
9	10.0	15.3	60.0	41.6	92.1	**86.8**	66.8	59.5	80.5	61.0	64.2	39.8	**93.2**	71.7
10	16.2	64.7	41.3	82.1	57.3	86.6	34.0	91.0	60.8	87.6	54.3	94.8	**68.6**	**98.2**
11	78.2	78.9	68.7	85.5	71.7	95.1	75.4	98.8	66.6	90.0	79.8	98.6	**90.7**	**99.3**
12	7.4	49.7	18.3	73.8	20.3	71.9	28.5	91.1	25.7	70.3	25.5	91.7	**47.7**	**94.6**
13	31.1	94.4	44.1	94.2	72.1	99.9	55.4	98.4	69.1	99.1	48.5	92.8	**97.8**	**99.9**
14	87.6	95.3	85.8	96.0	90.2	95.8	80.1	98.2	86.0	98.5	96.0	99.7	**98.9**	**99.9**
15	13.0	43.5	23.6	75.4	30.2	94.3	36.6	91.6	34.5	87.4	38.1	93.5	**51.0**	**99.7**
16	73.2	84.9	26.1	93.0	83.2	96.3	36.5	**97.3**	82.9	94.1	64.1	75.2	**95.3**	93.0
OA	49.4	74.4	52.1	84.5	59.7	92.4	56.2	95.0	61.0	87.4	63.6	95.3	**79.6**	**98.3**
AA	38.1	64.7	43.8	77.5	59.2	89.9	54.2	90.9	62.4	83.3	57.6	82.2	**80.7**	**95.3**
Kappa	40.3	70.5	45.7	84.0	53.8	91.4	49.6	94.3	55.6	85.6	57.8	94.7	**76.5**	**98.1**

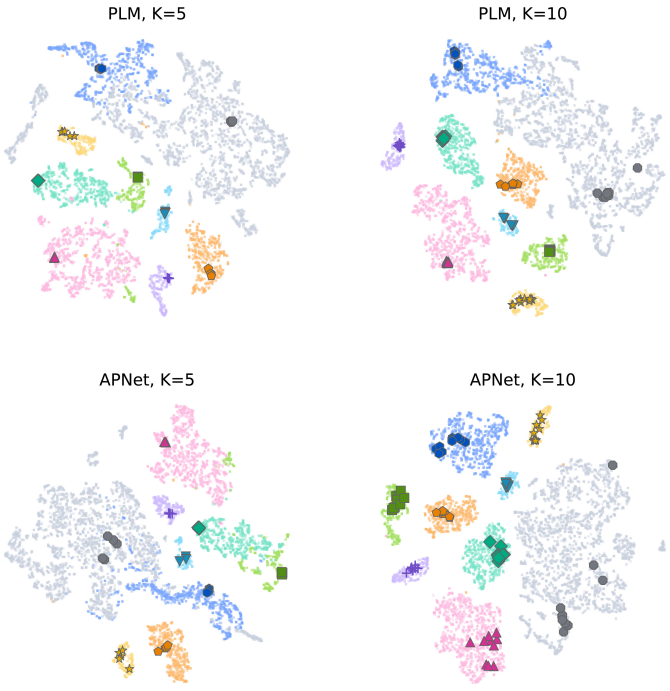


Fig. 13. Visualizations of the feature distributions for the baseline method (first row) and APNet (second row) on the University of Pavia dataset. Dark markers represent training samples, and light dots represent testing samples.

as shown in Fig. 13, where "PLM" denotes APNet with $\lambda = 0$. The dark markers represent training samples, the light dots represent testing samples, and the class of each sample is distinguished by its color. It can be seen in the first row of Fig. 13 that the training samples are biased with respect to the testing samples. The samples from some classes overlap with each other in the feature space. This indicates that the PLM alone cannot handle the distribution bias, which prevents it from learning discriminative features and limits its generalization ability. In the second row of Fig. 13, it can be seen that, with the introduction of the AAM, the training samples are more uniformly distributed in the testing feature space, i.e., the training and testing features are well aligned. In this way, the training samples become more representative of the testing samples, and consequently, the model trained on the training set can be more easily generalized to the testing set.

5) Effect of λ: In this section, we present experiments conducted to study the effect of the hyperparameter λ defined in (5) and (7). λ is a regularization factor that influences the balance between the alignment of the feature distributions and the training of classifiers. It can be seen in Fig. 14 that, when $K = 2$, the performance increases as λ increases from 0.0 to 1.0. However, if λ continues to increase, the performance begins to drop. When $K = 5$, the optimal λ value is lower, i.e., 0.5. This is because the bias decreases as K increases. Consequently, we need to reduce λ to allow the model to pay more attention to training classifiers on the training set.

6) Effect of the Number of Iterations of PLM and AAM Training: Fig. 15 shows the effect of the number of iterations of PLM and AAM training. The experiments were conducted on the Indian Pines dataset with $K = 2$ and $K = 5$. With only one iteration of PLM and AAM training, APL already achieves considerable performance. When the number of iterations is increased, the performance in terms of OA, AA, and Kappa further improves. The optimal number of iterations is 5 for both $K = 2$ and $K = 5$. When further iterations are performed, the performance will slightly decrease.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
COMPARISON OF INFERENCE TIME (SECONDS) WITH STATE-OF-THE-ART METHODS

Data set	SVM [12]	3D-CNN [22]	pResNet [23]	ASSMN [31]	CBW [32]	FreeNet [24]	APNet
University of Pavia	2.30	3.32	6.26	9.95	7.17	0.092	**0.055**
Indian Pines	0.95	0.48	2.72	3.64	0.89	0.024	**0.011**
Salinas	3.12	2.64	11.40	12.72	4.84	0.089	**0.037**
KSC	6.44	6.57	2.19	1.72	12.88	0.216	**0.058**

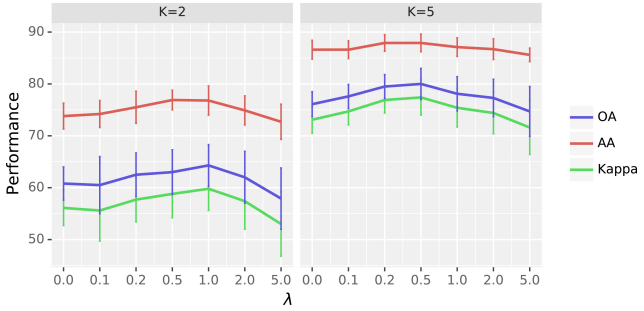


Fig. 14. Effects of λ on the Indian pines dataset with $K = 2$ and $K = 5$.

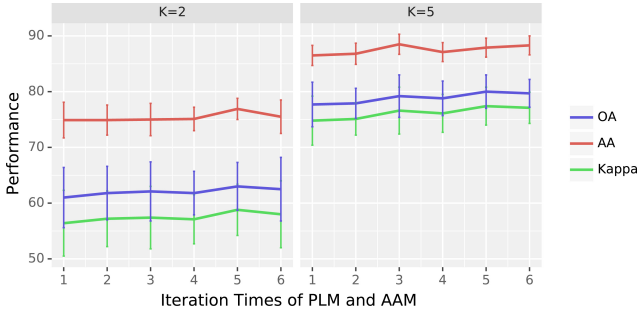


Fig. 15. Effects of the number of iterations of PLM and AAM training on the Indian pines dataset with $K = 2$ and $K = 5$.

TABLE VIII
PERFORMANCE COMPARISON OF APL COMBINED WITH DIFFERENT FEATURE EXTRACTORS ON THE INDIAN PINES DATASET

Method	K=2			K=5		
	OA	AA	Kappa	OA	AA	Kappa
pResNet	41.6	56.2	35.3	58.3	70.4	53.1
CBW	47.3	60.6	41.1	59.4	73.0	54.5
pResNet-APL	52.0	68.1	46.9	66.3	79.9	62.3
CBW-APL	49.5	64.0	44.0	63.3	76.9	59.1
APNet	**63.1**	**75.5**	**58.5**	**75.8**	**85.9**	**72.7**

7) Inference Time Comparison: Table VII shows the inference times of the proposed APNet method and the methods considered for comparison. APNet is much faster than 3-D-CNN, pResNet, ASSMN, and CBW in inferring the classes of testing pixels on all four datasets. APNet is also slightly faster than FreeNet while achieving a significant performance improvement.

8) Effects of Different Feature Extractors: Table VIII shows the results of combining APL with the feature extractors of pResNet and CBW when $K = 2$ and $K = 5$. It can be seen that pResNet-APL and CBW-APL consistently outperform pResNet and CBW, thus demonstrating that APL can be successfully combined with other feature extractors. Nevertheless, the proposed APNet method outperforms pResNet-APL and CBW-APL, which indicates that APNet is better at extracting spatial and spectral features than pResNet and CBW.

VI. CONCLUSION

In this article, a novel method named APL is proposed to alleviate overfitting while aligning the feature distributions between the training and testing sets for HSI classification. APL is able to reduce the feature distribution bias between the training and testing sets and improve the generalization ability of the learned model, as validated through theoretical analysis. The APNet architecture is further proposed to combine APL with DL in an efficient end-to-end manner. APNet achieves significant performance improvement in comparison with the baseline method and outperforms state-of-the-art methods on four publicly available datasets. The proposed APL and APNet provide new insights regarding generic classification tasks with small training sets in geoscience.

REFERENCES

[1] X. Yang and Y. Yu, "Estimating soil salinity under various moisture conditions: An experimental study," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 5, pp. 2525–2533, May 2017.
[2] N. Yokoya, J. C.-W. Chan, and K. Segl, "Potential of resolution-enhanced hyperspectral data for mineral mapping using simulated EnMAP and Sentinel-2 images," *Remote Sens.*, vol. 8, no. 3, p. 172, 2016.
[3] S. Li, R. Dian, L. Fang, and J. M. Bioucas-Dias, "Fusing hyperspectral and multispectral images via coupled sparse tensor factorization," *IEEE Trans. Image Process.*, vol. 27, no. 8, pp. 4118–4130, Aug. 2018.
[4] S. Zhang, J. Li, Z. Wu, and A. Plaza, "Spatial discontinuity-weighted sparse unmixing of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 10, pp. 5767–5779, Oct. 2018.
[5] T. V. Bandos, L. Bruzzone, and G. Camps-Valls, "Classification of hyperspectral images with regularized linear discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 3, pp. 862–873, Mar. 2009.
[6] A. Villa, J. A. Benediktsson, J. Chanussot, and C. Jutten, "Hyperspectral image classification with independent component discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 12, pp. 4865–4876, Dec. 2011.
[7] G. Licciardi, P. R. Marpu, J. Chanussot, and J. A. Benediktsson, "Linear versus nonlinear PCA for the classification of hyperspectral data based on the extended morphological profiles," *IEEE Geosci. Remote Sens. Lett.*, vol. 9, no. 3, pp. 447–451, May 2012.
[8] J. A. Benediktsson, J. A. Palmason, and J. R. Sveinsson, "Classification of hyperspectral data from urban areas based on extended morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 480–491, Mar. 2005.
[9] Y. Qian, M. Ye, and J. Zhou, "Hyperspectral image classification based on structured sparse logistic regression and three-dimensional wavelet texture features," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 4, pp. 2276–2291, Apr. 2013.
[10] W. Li, C. Chen, H. Su, and Q. Du, "Local binary patterns and extreme learning machine for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 7, pp. 3681–3693, Jul. 2015.
[11] S. Jia, K. Wu, J. Zhu, and X. Jia, "Spectral-spatial Gabor surface feature fusion approach for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1142–1154, Feb. 2019.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

[12] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[13] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Semisupervised hyperspectral image segmentation using multinomial logistic regression with active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 11, pp. 4085–4098, Nov. 2010.

[14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. 26th Annu. Conf. Neural Inf. Process. Syst.*, vol. 25, P. L. Bartlett, F. C. N. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds., Lake Tahoe, NV, USA, 2012, pp. 1106–1114.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.

[16] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. Washington, DC, USA: IEEE Computer Society, Jun. 2018, pp. 7132–7141.

[17] S. Li, W. Song, L. Fang, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Deep learning for hyperspectral image classification: An overview," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6690–6709, Sep. 2019.

[18] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[19] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.

[20] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4959–4962.

[21] X. Li, M. Ding, and A. Pižurica, "Deep feature fusion via two-stream convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2615–2629, Apr. 2020.

[22] A. B. Hamida, A. Benoit, P. Lambert, and C. B. Amar, "3-D deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.

[23] M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, J. Plaza, A. J. Plaza, and F. Pla, "Deep pyramidal residual networks for spectral-spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 740–754, Feb. 2019.

[24] Z. Zheng, Y. Zhong, A. Ma, and L. Zhang, "FPGA: Fast patch-free global learning framework for fully end-to-end hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5612–5626, Aug. 2020.

[25] R. Hang, F. Zhou, Q. Liu, and P. Ghamisi, "Classification of hyperspectral images via multitask generative adversarial networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 2, pp. 1424–1436, Feb. 2021.

[26] J. Snell, K. Swersky, and R. S. Zemel, "Prototypical networks for few-shot learning," in *Proc. 30th Conf. Neural Inf. Process. Syst. (NeurIPS)*, I. Guyon *et al.*, Eds., 2017, pp. 4077–4087.

[27] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2018, pp. 1199–1208.

[28] A. Banerjee, S. Merugu, I. S. Dhillon, and J. Ghosh, "Clustering with Bregman divergences," *J. Mach. Learn. Res.*, vol. 6, no. 4, pp. 1705–1749, 2005.

[29] J. Li, J. Bioucas-Dias, and A. Plaza, "Spectral–spatial hyperspectral image segmentation using subspace multinomial logistic regression and Markov random fields," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 809–823, Mar. 2012.

[30] Y. Zhong and L. Zhang, "An adaptive artificial immune network for supervised classification of multi-/hyperspectral remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 894–909, Mar. 2012.

[31] D. Wang, B. Du, L. Zhang, and Y. Xu, "Adaptive spectral-spatial multiscale contextual feature extraction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2461–2477, Mar. 2020.

[32] L. Zhao, J. Yi, X. Li, W. Hu, J. Wu, and G. Zhang, "Compact band weighting module based on attention-driven for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Feb. 4, 2021, doi: 10.1109/TGRS.2021.3053397.

[33] J. Feng *et al.*, "Attention multibranch convolutional neural network for hyperspectral image classification based on adaptive region search," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5054–5070, Jun. 2021.

[34] J. Feng *et al.*, "Generative adversarial networks based on collaborative learning and attention mechanism for hyperspectral image classification," *Remote Sens.*, vol. 12, no. 7, p. 1149, Apr. 2020.

[35] L. Mou, X. Lu, X. Li, and X. X. Zhu, "Nonlocal graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 12, pp. 8246–8257, Dec. 2020, doi: 10.1109/TGRS.2020.2973363.

[36] W. Li, Z. Xu, D. Xu, D. Dai, and L. Van Gool, "Domain generalization and adaptation using low rank exemplar SVMs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 5, pp. 1114–1127, May 2018.

[37] P. P. Busto, A. Iqbal, and J. Gall, "Open set domain adaptation for image and action recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 413–429, Feb. 2020.

[38] Z. Zhong, L. Zheng, Z. Luo, S. Li, and Y. Yang, "Invariance matters: Exemplar memory for domain adaptive person re-identification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. Washington, DC, USA: IEEE Computer Society, Jun. 2019, pp. 598–607.

[39] B. Fernando, A. Habrard, M. Sebban, and T. Tuytelaars, "Unsupervised visual domain adaptation using subspace alignment," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*. Washington, DC, USA: IEEE Computer Society, Dec. 2013, pp. 2960–2967.

[40] H. Lu *et al.*, "When unsupervised domain adaptation meets tensor representations," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*. Washington, DC, USA: IEEE Computer Society, Oct. 2017, pp. 599–608.

[41] A. Samat, C. Persello, P. Gamba, S. Liu, J. Abuduwaili, and E. Li, "Supervised and semi-supervised multi-view canonical correlation analysis ensemble for heterogeneous domain adaptation in remote sensing image classification," *Remote Sens.*, vol. 9, no. 4, p. 337, 2017.

[42] Y. Qin, L. Bruzzone, B. Li, and Y. Ye, "Cross-domain collaborative learning via cluster canonical correlation analysis and random Walker for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 6, pp. 3952–3966, Jun. 2019.

[43] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," *J. Mach. Learn. Res.*, vol. 13, pp. 723–773, Mar. 2012.

[44] M. Long, Y. Cao, J. Wang, and M. I. Jordan, "Learning transferable features with deep adaptation networks," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, vol. 37, F. R. Bach and D. M. Blei, Eds., 2015, pp. 97–105.

[45] M. Long, H. Zhu, J. Wang, and M. I. Jordan, "Unsupervised domain adaptation with residual transfer networks," in *Proc. Adv. Neural Inf. Process. Syst.*, D. D. Lee, M. Sugiyama, U. von Luxburg, I. Guyon, and R. Garnett, Eds., Barcelona, Spain, 2016, pp. 136–144.

[46] C. Chen *et al.*, "Progressive feature alignment for unsupervised domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 627–636.

[47] K. Bousmalis, N. Silberman, D. Dohan, D. Erhan, and D. Krishnan, "Unsupervised pixel-level domain adaptation with generative adversarial networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. Washington, DC, USA: IEEE Computer Society, Jul. 2017, pp. 95–104.

[48] J. Hoffman *et al.*, "Cycada: Cycle-consistent adversarial domain adaptation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 80, J. G. Dy and A. Krause, Eds., 2018, pp. 1994–2003.

[49] M. Ye, Y. Qian, J. Zhou, and Y. Y. Tang, "Corrections to 'dictionary learning-based feature-level domain adaptation for cross-scene hyperspectral image classification,"' *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 5, pp. 3002–3003, May 2018.

[50] X. Ma, X. Mou, J. Wang, X. Liu, H. Wang, and B. Yin, "Cross-data set hyperspectral image classification based on deep domain adaptation," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 12, pp. 10164–10174, Dec. 2019.

[51] X. Zhou and S. Prasad, "Deep feature alignment neural networks for domain adaptation of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 10, pp. 5863–5872, Oct. 2018.

[52] Y. Qin, L. Bruzzone, and B. Li, "Tensor alignment based domain adaptation for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 11, pp. 9290–9307, Nov. 2019.

[53] L. Bruzzone, M. Chi, and M. Marconcini, "Transductive SVMs for semisupervised classification of hyperspectral data," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Seoul, South Korea, Jul. 2005, p. 5.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.

[54] I. Dópido, J. Li, P. R. Marpu, A. Plaza, J. M. B. Dias, and J. A. Benediktsson, "Semisupervised self-learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 7, pp. 4032–4044, Jul. 2013.
[55] H. Wu and S. Prasad, "Semi-supervised deep learning using pseudo labels for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 27, no. 3, pp. 1259–1270, Mar. 2018.
[56] Y. Zhan, D. Hu, Y. Wang, and X. Yu, "Semisupervised hyperspectral image classification based on generative adversarial networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 2, pp. 212–216, Feb. 2018.
[57] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5046–5063, Sep. 2018.
[58] S. Zhou, Z. Xue, and P. Du, "Semisupervised stacked autoencoder with cotraining for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 6, pp. 3813–3826, Jun. 2019.
[59] P. Sellars, A. I. Avilés-Rivero, and C.-B. Schönlieb, "Superpixel contracted graph-based learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 6, pp. 4180–4193, Jun. 2020.
[60] T. Kohonen, *Self-Organizing Maps* (Springer Series in Information Sciences), vol. 30. Berlin, Germany: Springer, 1995.
[61] C.-L. Liu, H. Sako, and H. Fujisawa, "Effects of classifier structures and training regimes on integrated segmentation and recognition of handwritten numeral strings," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 26, no. 11, pp. 1395–1407, Nov. 2004.
[62] S.-W. Lee and H.-H. Song, "Optimal design of reference models for large-set handwritten character recognition," *Pattern Recognit.*, vol. 27, no. 9, pp. 1267–1274, Sep. 1994.
[63] C.-L. Liu, I.-J. Eim, and J. H. Kim, "High accuracy handwritten Chinese character recognition by improved feature matching method," in *Proc. 4th Int. Conf. Document Anal. Recognit. (ICDAR)*. Washington, DC, USA: IEEE Computer Society, 1997, pp. 1033–1037.
[64] J. Liu, L. Song, and Y. Qin, "Prototype rectification for few-shot learning," in *Proc. 16th Eur. Conf.*, vol. 12346. Glasgow, U.K.: Springer, Aug. 2020, pp. 741–756.
[65] B. Yang, C. Liu, B. Li, J. Jiao, and Q. Ye, "Prototype mixture models for few-shot semantic segmentation," in *Proc. 16th Eur. Conf.*, vol. 12353. Glasgow, U.K.: Springer, Aug. 2020, pp. 763–778.
[66] D. Cerra, R. Müller, and P. Reinartz, "A classification algorithm for hyperspectral images based on synergetics theory," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 5, pp. 2887–2898, May 2013.
[67] M. G. Asl, M. R. Mobasheri, and B. Mojaradi, "Unsupervised feature selection using geometrical measures in prototype space for hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 7, pp. 3774–3787, Jul. 2014.
[68] Z. Xue, P. Du, J. Li, and H. Su, "Simultaneous sparse graph embedding for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 11, pp. 6114–6133, Jun. 2015.
[69] C. Zhang, J. Yue, and Q. Qin, "Global prototypical network for few-shot hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4748–4759, 2020.
[70] H. Tang, Y. Li, X. Han, Q. Huang, and W. Xie, "A spatial–spectral prototypical network for hyperspectral remote sensing image," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 1, pp. 167–171, Jan. 2020.
[71] B. Xi *et al.*, "Deep prototypical networks with hybrid residual attention for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 3683–3700, 2020.
[72] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Mach. Learn.*, vol. 79, nos. 1–2, pp. 151–175, 2010.
[73] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 21–37.
[74] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020.
[75] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. Honolulu, HI, USA: IEEE Computer Society, Jul. 2017, pp. 936–944.
[76] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 2605, pp. 2579–2605, 2008.



Shuai Wang (Graduate Student Member, IEEE) received the B.S. degree in mathematics from Hefei University of Technology, Hefei, China, in 2009, the B.S. degree in resource exploration engineering from China University of Geosciences, Wuhan, China, in 2013, and the M.S. degree in mineral resource prospecting and exploration from China University of Geosciences, Beijing, China, in 2016, where he is pursuing the Ph.D. degree.

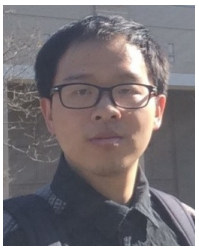
His research interests include remote sensing and machine learning in geoscience.



Bo Du (Senior Member, IEEE) received the Ph.D. degree in photogrammetry and remote sensing from the State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing, Wuhan University, Wuhan, China, in 2010.

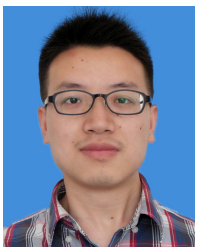
He is a Professor with the School of Computer Science, Wuhan University, and the Institute of Artificial Intelligence, Wuhan University. He is also the Director of the National Engineering Research Center for Multimedia and Software, Wuhan University. He has published more than 80 research articles in IEEE TRANSACTIONS ON IMAGE PROCESSING (TIP), IEEE TRANSACTIONS ON CYBERNETICS (TCYB), IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE (TPAMI), IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING (TGRS), IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (JSTARS), and IEEE GEOSCIENCE AND REMOTE SENSING LETTERS (GRSL). Thirteen of them are Essential Science Indicators (ESI) hot articles or highly cited articles. His research interests include pattern recognition, hyperspectral image processing, and signal processing.

Dr. Du regularly serves as a Senior Program Committee (PC) Member of the International Joint Conferences on Artificial Intelligence (IJCAI) and the Association for the Advancement of Artificial Intelligence (AAAI). He also serves as a reviewer for 20 Science Citation Index (SCI) magazines, including IEEE TPAMI, TCYB, TGRS, TIP, JSTARS, and GRSL. He received the Highly Cited Researcher by the Web of Science Group in 2019 and 2020, received the IEEE Geoscience and Remote Sensing Society (GRSS) 2020 Transactions Prize Paper Award, received the IJCAI Distinguished Paper Prize, was the IEEE Data Fusion Contest Champion, and received the IEEE Workshop on Hyperspectral Image and Signal Processing Best Paper Award in 2018. He has served as an Area Chair for the International Conference on Pattern Recognition (ICPR). He serves as an Associate Editor for *Neural Networks*, *Pattern Recognition*, and *Neurocomputing*.



Dingwen Zhang (Member, IEEE) received the Ph.D. degree from Northwestern Polytechnical University, Xi'an, China, in 2018.

From 2015 to 2017, he was a Visiting Scholar with the Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, USA. He is a Professor with the School of Automation, Northwestern Polytechnical University. His research interests include computer vision and machine learning.



Fang Wan (Member, IEEE) received the B.S. degree from Wuhan University, Wuhan, China, in 2013, and the Ph.D. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2019.

Since then, he has been a Post-Doctoral Researcher with the School of Computer Sciences, University of Chinese Academy of Sciences. He has published more than 15 articles in refereed conference proceedings and journals, including IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE (TPAMI), IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS (TNNLS), Conference on Computer Vision and Pattern Recognition (CVPR), International Conference on Computer Vision (ICCV), and Association for the Advancement of Artificial Intelligence (AAAI). His research interests include computer vision and machine learning.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:50:26 UTC from IEEE Xplore. Restrictions apply.